Real-World Application & Future Pathways

Real-World Application & Future Pathways

From Coding Foundations to Web, AI, and Digital Product Creation

Learning Outcomes

By following this learning pathway, students will be able to:

- build a strong foundation in coding logic, including sequence, loops, conditionals, debugging, and problem-solving
- create interactive projects in Scratch, moving from guided tasks to independent design and system thinking
- understand how games and digital systems work through input, logic, feedback, score, progression, and user interaction
- use AI responsibly as a support tool for thinking, creativity, planning, and communication rather than as a replacement for their own ideas
- translate ideas into structured systems using flowcharts, planning methods, and step-by-step logic before building projects
- transition from block-based coding to text-based programming in Python with confidence
- build Python programs using variables, input, conditions, functions, loops, and system design concepts
- develop computational thinking, creativity, communication, and presentation skills through project-based learning and final showcases

Course Structure (Step by Step)

Step	Stage	What Students Learn	Why They Learn It	Why It Matters
1	Scratch Foundations	Students begin with algorithm, sequence, loops, conditionals, debugging, and basic Scratch use. They learn how code works through guided animations and mini-games.	This stage builds the basic language of programming. Before students can create freely, they need to understand how instructions, repetition, and decisions work.	It gives students a solid coding foundation and helps them think step by step instead of guessing.

2	Scratch Creator	Students move from following instructions to designing their own interactive stories and projects. They begin using variables, timing, effects, broadcast, and Input → Process → Output thinking.	After learning the basics, students must learn how to combine ideas into a complete system. This helps them shift from “doing tasks” to “creating with purpose.”	It develops independence, creativity, and the ability to explain how a project works, not just build it.
3	Scratch Game Developer	Students build structured games such as Pong, Catch, and Clicker systems. They learn scoring, collision detection, events, progression, and simple AI-generated asset integration.	Games are a strong way to teach system logic because students can clearly see how input, rules, feedback, and progression connect.	It helps students understand that coding is not just about blocks, but about designing complete systems that behave logically.
4	Scratch Advanced Game Systems	Students learn deeper game architecture such as clones, physics movement, game states, UI systems, lists, and multi-system integration.	Once students can build small games, they need to understand how bigger systems are organized. This is where coding becomes architecture, not only creation.	It prepares students for advanced project building, debugging, and structured design thinking.
5	AI Explorer	Students learn what AI is, how it works, how to prompt clearly, how to evaluate outputs, and how to use AI safely and responsibly. They also use AI-supported tools for slides and project communication.	Students need to understand AI as a system, not as magic. They must learn how to question outputs, protect information, and use AI as support.	This builds digital responsibility, critical thinking, and communication skills that are important in modern learning.
6	AI to Scratch System Builder	Students use AI for idea generation, then convert those ideas into flowcharts and structured Scratch systems. They learn to plan before coding and defend their logic through	This stage teaches that good projects begin with planning. Students learn that AI can help generate ideas, but the real	It strengthens system thinking, planning, reasoning, and project ownership.

		explanation and presentation.	work is turning ideas into logical systems.	
7	Python Foundations	Students transition from Scratch to Python by learning print(), variables, data types, input(), and basic calculations. They begin understanding text-based coding through Input → Process → Output.	After block coding, students need a bridge into real syntax. Python gives them a clean way to understand how written code works.	It prepares students for more advanced programming and gives them confidence in text-based problem solving.
8	Python Decision Systems	Students learn comparison operators and if / elif / else structures to build programs that make decisions. They create interactive programs such as quizzes, checkers, and logic systems.	Coding becomes more powerful when programs can respond differently based on conditions. Students must learn how systems “decide.”	It develops logical reasoning and helps students design smarter, more useful programs.
9	Python System & Function Builder	Students learn functions, parameters, system flow, while loops, and modular thinking. They begin designing programs that are more organized and reusable.	Once students know basic logic, they need structure. Functions help them break programs into meaningful parts instead of writing everything in one place.	It prepares them for real software thinking, where clarity, reuse, and system design matter.
10	Python AI System & Intelligence Builder	Students explore the difference between rule-based systems and AI-like systems. They use data, memory, patterns, prediction, and decision-making to create adaptive programs.	This stage helps students understand that intelligence in systems comes from data and patterns, not randomness.	It gives them an early foundation in AI thinking, data-driven logic, and responsible use of intelligent systems.

Step-by-Step Learning Logic

First, students learn coding fundamentals because they need a strong base in sequence, logic, loops, and debugging before they can build meaningful projects.

Next, they move into creation and game design so they can apply logic in more visible and engaging systems. This helps them understand how rules, interaction, and feedback work in real projects.

Then, they are introduced to AI and system planning so they do not see AI as a shortcut, but as a tool that must be guided, checked, and used responsibly. Flowcharts and structured planning help students think before they build.

Finally, they transition into Python so they can move from visual logic to real programming syntax, then into structured systems, and eventually into AI-style logic using data and prediction.

This means the learning journey follows a clear progression:

Understand basic logic → create interactive systems → design bigger structures → think critically about AI → plan systems clearly → code in Python → build intelligent programs.

Real-World Application & Extension Path

Why This Learning Matters Beyond Coding

This learning pathway is not designed only to teach coding skills, but to prepare students to **build real digital products and systems** used in the real world.

Each stage connects directly to modern technology fields such as:

- Web Development (websites, applications, platforms)
- Game Development (interactive systems, logic design)
- AI-Assisted Creation (content, automation, decision systems)
- Software Engineering (structured systems, modular thinking)

Students are not just learning “how to code”
They are learning **how systems work and how to build them intentionally**

From Coding → Web Development

After completing the Scratch → AI → Python progression, students are ready to move into **Web Development**.

What transfers directly:

What Students Learned	How It Applies to Web
Input → Process → Output	Form handling, user interaction
Game Flow (Start → Play → End)	Page flow (Home → Action → Result)
Variables & Data	JavaScript variables & state
Conditionals (if/else)	UI logic (show/hide, validation)
System Thinking	Full web application structure
Debugging	Fixing UI / logic bugs

👉 This means:
Students are **already thinking like developers**
before they even start HTML / CSS / JavaScript

Web Learning Direction (Next Step)

After this curriculum, students can transition into:

1. HTML (Structure)

- Build the skeleton of a website
- Understand content organization
- Learn how information is displayed

2. CSS (Design)

- Control layout, color, spacing
- Understand visual hierarchy
- Improve user experience (UX)

3. JavaScript (Logic & Interaction)

- Apply real programming logic to websites
- Handle clicks, forms, and dynamic content
- Build interactive web systems

👉 This directly extends from:
Scratch logic → Python logic → JavaScript logic

From Projects → Digital Products

Students move from:

- “making projects for fun”
  → to
- “building products with purpose”

Examples of Real Applications:

- Personal portfolio website
- Simple booking system
- Interactive learning website
- Game hosted on web
- AI-assisted content generator

👉 This is where students start thinking like:
Creators, not just learners

AI Integration in Real-World Use

Students will apply AI in practical ways:

- Generate content (text, images, ideas)
- Assist in coding (but not replace thinking)
- Improve productivity and creativity
- Support project planning and design

Key mindset:
AI is a tool, not the thinker

Future Pathways

After completing this full journey, students can progress into:

1. AI Creator Path

- Build web + AI integrated products
- Use APIs and automation
- Create real-world digital solutions

2. AI Specialist Path

- Data thinking & analysis
- Machine learning fundamentals
- Model understanding and prediction systems

3. Software / Web Developer Path

- Full-stack development
- Backend systems & databases
- Real application deployment

Final Outcome (Big Picture)

By the end of the full pathway, students will be able to:

- Think in systems (not just code)
- Design and build real digital experiences
- Use AI responsibly and effectively
- Transition across platforms (Scratch → Python → Web)
- Communicate and present their ideas clearly

👉 Most importantly:

Students become
builders of technology, not just users of it

Tools & Technologies

🔹 Tools & Technologies (AI-Focused Learning Environment)

Overview

This program integrates carefully selected AI tools to support structured thinking, creativity, and real-world application.
Students do not simply “use AI” — they learn how to **understand, control, and apply AI as a thinking tool**, not a replacement.

The tools are chosen to align with the CIY philosophy:
Understand → Control → Create → Build → Present

🔹 Core AI Tools

1. ChatGPT (Primary AI System)

Purpose: Thinking, Prompting, Writing, Idea Generation

Students use ChatGPT as their main AI interaction tool to:

- Understand how AI processes input → output
- Practice structured prompting (RACE Framework)
- Generate ideas, explanations, and structured content
- Improve writing and communication clarity

Learning Focus:

- AI Thinking (Input → Data → Pattern → Output)
- Prompt Engineering
- Output Evaluation (Is this correct? Why?)

---

2. Gemini (Comparative AI System)

Purpose: AI Comparison & Critical Thinking

Students use Gemini to compare outputs with ChatGPT:

- Same prompt → different result
- Analyze accuracy, style, and reliability
- Understand that AI is not always “correct”

Learning Focus:

- Critical Thinking
- AI Limitations
- Bias & Variation in AI Systems

---

3. DeepAI (AI Image Generation)

Purpose: Visual Creation & Prompt Control

Students generate images using AI to understand how prompts affect visuals:

- Character creation
- Scene generation
- Style experimentation

Learning Focus:

- Prompt structure for visuals (SASE Framework)
- Cause → Effect in prompt design
- Creative direction using AI

🔹 AI Output & Presentation Tools

4. Gamma (AI Presentation Tool)

Purpose: Slide Creation & Idea Structuring

Students use Gamma to:

- Turn ideas into structured presentations
- Generate slide content with AI support
- Organize storytelling and flow

Learning Focus:

- Communication & Presentation
- Structuring information clearly
- Turning ideas into visual narratives

---

5. Canva (Design & Visual Communication)

Purpose: Design, Layout, and Visual Enhancement

Students refine their presentations and visuals:

- Layout design
- Visual storytelling
- Enhancing AI-generated content

Learning Focus:

- Visual communication
- Design thinking
- Human creativity over AI output

🔹 Thinking & Control Frameworks

6. Prompt Template Sheet (RACE Framework)

Purpose: Structured AI Communication

Students learn to control AI using:

- **Role** – Who AI should act as
- **Action** – What AI should do
- **Context** – Background information
- **Expectation** – Desired output

Learning Focus:

- Clear thinking before asking
- Structured communication
- Reducing random or unclear outputs

---

7. SASE Framework (Image Prompting)

Purpose: Visual Prompt Engineering

Students learn how to control image outputs using:

- **Subject**
- **Action**
- **Setting**
- **Style**

Learning Focus:

- Visual logic
- Creative direction
- Precision in prompts

🔹 Supporting Learning Tools (Non-AI)

8. Scratch / Code.org (Optional Integration)

Purpose: System Thinking & Logic Application

Used when connecting AI → system building:

- Input → Process → Output
- Game logic and interaction systems

🔹 Key Teaching Principle

All tools are used under one core rule:

> **AI is a support tool, not a replacement for thinking.**

Students are trained to:

- Think before prompting
- Question AI outputs
- Take ownership of their work
- Use AI responsibly and safely

🔹 Summary (For Proposal / Slide Use)

AI Tools Used in This Program:

- ChatGPT (Primary AI Thinking Tool)
- Gemini (AI Comparison & Critical Thinking)
- DeepAI (Image Generation)
- Gamma (AI Presentation)
- Canva (Design & Visual Communication)

Frameworks:

- RACE (Text Prompting)
- SASE (Image Prompting)

BLOCK CODING

01 Scratch
Foundations
Loops · Sequence · Debug

→

02 Scratch
Creator
Variables · Broadcast · IPO

→

03 Scratch
Game Dev
Pong · Scoring · Events

→

04 Scratch
Advanced
Clones · Physics · UI

↓

AI & SYSTEM THINKING

05 AI Explorer
How AI works · Prompt engineering · Safe use
ChatGPT · Gemini · RACE Framework

→

06 AI → Scratch Builder
Idea gen → Flowchart → Scratch system
Plan before code · Defend your logic

↓

TEXT-BASED PROGRAMMING

07 Python
Foundations
Print · Variables
input() · IPO

→

08 Python
Decision Systems
if / elif / else
Quizzes · Logic

→

09 Python
System Builder
Functions · Loops
Modular design

→

10 Python AI
Intelligence
Data · Patterns
Prediction logic

↓

FUTURE PATHWAYS

AI Creator
Web + AI products
APIs · Automation

AI Specialist
Data thinking
ML fundamentals

Web Developer
HTML · CSS · JS
Full-stack apps

KEY TOOLS & FRAMEWORKS

ChatGPT · Gemini · DeepAI · Gamma · Canva · Scratch · Python · RACE · SASE

Learning logic
Understand logic → Create systems → Think about AI → Code in Python → Build intelligent programs

STUDENTS BECOME

System thinkers
not just coders

Responsible AI
users & evaluators

Digital creators
with real products

Confident presenters
who defend their ideas

Scratch

LEVEL 1: Scratch Foundations

Code.org + Guided Scratch Projects
Duration: 8 Weeks
Presented by: CIY.CLUB

Learning Outcomes

By the end of this course, students will be able to:

- Understand core coding concepts: **algorithm, sequence, loops, conditionals, and debugging**
- Apply computational thinking to solve problems step-by-step
- Use Scratch interface confidently (sprites, scripts, events, motion)
- Follow structured instructions to build animations and simple games
- Identify and fix errors using basic debugging strategies
- Create interactive animations and mini-games based on guided tasks
- Explain the logic behind their code in a clear and simple way

Course Highlights

Strong Coding Foundation
Students build a solid understanding of fundamental programming concepts through structured learning.

Blended Learning Approach
Combines **unplugged activities**, Code.org, and Scratch to reinforce understanding from multiple angles.

Guided Project-Based Learning
Students learn by building real projects step-by-step with coach support.

Confidence Building
Focus on helping students feel comfortable with coding through clear instructions and progressive challenges.

Computational Thinking Development
Students learn how to think logically, break down problems, and build solutions systematically.

Course Structure

Week	Theme	Focus	Activity	Outcome
1	Algorithm & Sequence	Step-by-step thinking	Unplugged + Code.org C	Students understand algorithms and correct sequencing
2	Loops & Patterns	Reduce repetition	Code.org C	Students can apply loops to simplify repeated actions
3	Conditionals & Debugging	Decision making	Code.org C	Students use if/then logic and basic debugging
4	Intro to Scratch / Project 1	Interface, Sprites / Space Talk	Scratch Exploration + Guided Project	Students use Scratch basics and create simple animation
5	Project 2	Catch the Bus	Guided Project	Students apply movement and timing logic
6	Project 3	Find the Bug	Guided Project	Students develop interaction and control systems
7	Project 4 + Mini Project	Silly Eyes + Coding in Use	Guided + Small Project	Students combine skills and begin explaining logic
8	Mini Project	Coding in Use	Small Project	Students debug, refine, and explain their project

Key Projects (Level 1)

- **Space Talk** → Animation + speech
- **Catch the Bus** → Movement + timing logic
- **Find the Bug** → Interaction and control
- **Silly Eyes** → Mouse-based interaction
- **Final Mini Project** → Combine all learned concepts

The Final Week (Week 8) Will Include

- **Mini Project Completion**
  Students finalize their own small project by combining all learned concepts
- **Logic Explanation**
  Students explain how their project works (sequence, loops, conditionals)
- **Debugging Practice**
  Students identify and fix errors in their own or peer projects
- **Reflection Activity**
  Students reflect on what they learned throughout the course
- **Celebration Activity**
  Fun group activity to reinforce confidence and enjoyment

Certification & Recognition

All students who successfully complete the **Level 1: Scratch Foundations Program** will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Scratch Programming Fundamentals** – Understanding and applying core coding concepts
- **Computational Thinking** – Breaking down problems and building logical solutions
- **Problem-Solving & Debugging** – Identifying and correcting errors in code
- **Creative Coding** – Building animations and simple interactive projects
- **Confidence in Coding** – Developing the ability to explain their own logic

The certification represents a strong foundation in coding and prepares students to progress into **Level 2: Scratch Creator**, where they move from guided learning to independent project creation.

LEVEL 2: Scratch Creator

Creative Design & Independent Projects
Duration: 8 Weeks
Presented by: CIY.CLUB

Learning Outcomes

By the end of this course, students will be able to:

- Design and develop their own interactive Scratch projects from concept to completion
- Apply **system thinking (Input → Process → Output)** to structure their programs
- Use advanced Scratch features such as variables, timing, effects, and broadcast
- Create interactive stories and animations with user control and multiple scenes
- Plan, build, and refine a project independently with guided coaching
- Explain the logic and structure behind their projects clearly
- Present their work confidently to peers, parents, and coaches

Course Highlights

Creative Design & Storytelling
Students move beyond following instructions to designing their own stories, games, and interactive experiences.

System Thinking Approach
Focus on understanding how programs work as systems (Input → Process → Output), not just writing code.

Project-Based Learning (PBL)
Students build meaningful projects that reflect real-world thinking and creativity.

Independent Creation with Coaching Support
Students are guided by coaches but encouraged to think, design, and solve problems independently.

Presentation & Communication Skills
Students learn how to explain their ideas, logic, and creative decisions effectively.

Course Structure

Week	Theme	Focus	Activity	Outcome
1	Design Thinking	Idea → Story Flow → User Experience	Brainstorm + Story Mapping	Students develop a clear project concept and story flow
2	Advanced Scratch Tools	Variables / Timing / Effects / Broadcast	Guided Practice	Students apply advanced tools to control behavior and interaction
3	Project 1: Surprise! Animation	Storytelling + Timing + Visual Effects	Guided → Semi-guided	Students create a structured animated story with timing control
4	Project 2: I Made You a Book	Interactive Story + User Input	Semi-guided	Students build an interactive story with multiple scenes and controls
5	Project Planning	System Thinking (Input → Process → Output)	Planning Sheet + Coach Feedback	Students design their own project system with clear logic flow
6	Final Project Build (Part 1)	Independent Creation	Build	Students begin developing their own project based on their design
7	Final Project Build (Part 2)	Logic Explanation + Refinement	Build + Coaching	Students refine their project and explain

				how their system works
8	Final Presentation	Showcase + Communication	Presentation	Students confidently present their project and explain key concepts used

Key Projects (Level 2)

- **Surprise! Animation** → Storytelling, emotion, and timing
- **I Made You a Book** → Interactive storytelling with user control
- **Final Project** → Student-designed project (game / story / interactive system)

The Final Week (Week 8) Will Include

- **Project Showcase Presentation**
  Students present their final projects to peers, parents, and coaches
- **Project Explanation**
  Students explain how their project works, including logic, interactions, and design decisions
- **Reflection Activity**
  Students reflect on what they learned and how they improved
- **Celebration Activities**
  Fun group activities to reinforce confidence and enjoyment in coding

Certification & Recognition

All students who successfully complete the **Level 2: Scratch Creator Program** will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Creative Project Design** – Developing original ideas and transforming them into interactive projects
- **System Thinking & Logic** – Applying structured thinking (Input → Process → Output) in programming
- **Advanced Scratch Skills** – Using variables, timing, effects, and interaction systems
- **Independent Problem Solving** – Planning and building projects with increasing independence

- **Communication & Presentation** – Explaining ideas and presenting projects confidently

The certification represents a significant milestone, demonstrating that students have progressed from guided learning to **independent creation and system-level thinking** in coding.

LEVEL 3: Scratch Game Developer

AI Integration & Game System Design
Duration: 8 Weeks
Presented by: CIY.CLUB

Learning Outcomes

By the end of this course, students will be able to:

- Design and develop **complete interactive game systems** using Scratch
- Apply advanced coding concepts: **variables, collision detection, scoring systems, and event-driven logic**
- Integrate **AI-generated assets** (characters, backgrounds) into their games
- Build structured games such as **Pong, Catch Games, and Clicker Systems with upgrades**
- Understand and apply **game system design (Input → Logic → Feedback → Score → Progression)**
- Plan, develop, and refine a **personal game project independently**
- Debug and optimize their code for improved gameplay experience
- Present their project clearly, explaining both **game mechanics and coding logic**

Course Highlights

Game Development Focus
Students transition from simple projects to building **fully playable game systems with clear mechanics and rules**.

AI Integration
Students use AI tools to generate custom assets, enhancing creativity and ownership of their projects.

System Thinking in Games
Students learn how real games work through structured systems:
Input → Logic → Feedback → Score → Progression

Advanced Scratch Concepts
Focus on variables, collision detection, events, scoring systems, and game loops.

Independent Project Development
Students gradually move from guided learning to independent creation with coach support.

Presentation & Game Showcase
Students present their projects like real developers, explaining both design and logic.

Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Review & AI Image Introduction	Review core concepts + AI asset creation	Group Activity + Guided Practice	Students generate AI assets and successfully import them into Scratch
2	Pong Game	Collision detection / movement / scoring	Guided Project	Students build a Pong game with correct mechanics and scoring
3	Catch & Collect Game	Random spawning / scoring / events	Guided Project	Students create a game with dynamic scoring (gain/lose points)
4	Clicker Game System	Continuous input / variables / basic progression	Guided Project	Students build a clicker game with accurate scoring system
5	Clicker Shop & Upgrade System	Game economy / upgrades / UI navigation	Guided → Semi-guided	Students create a shop system that interacts with gameplay
6	Mini Project Planning & Build	Game design / system planning (Input → Logic → Output)	Planning + Build	Students design and begin building their own game system

7	Independent Project Development	Debugging / polishing / refinement	Build + Coaching	Students refine mechanics, fix bugs, and improve gameplay
8	Final Showcase & Presentation	Communication / system explanation	Presentation	Students present their game and explain logic and mechanics clearly

The Final Week (Week 8) Will Include

- **Final Game Showcase**
  Students present their completed game to peers, parents, and coaches
- **Code & System Explanation**
  Students explain how their game works (logic, scoring, interactions, progression)
- **Reflection Activity**
  Students reflect on their learning journey and problem-solving process
- **Celebration Activities**
  Group activities to celebrate achievements and build confidence

Certification & Recognition

All students who successfully complete the **Level 3: Scratch Game Developer Program** will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Game Development Fundamentals** – Designing and building complete playable games
- **Advanced Coding Logic** – Applying variables, conditions, and event-driven systems
- **AI Integration** – Using AI-generated assets creatively within projects
- **System Thinking** – Understanding how game mechanics and logic systems connect
- **Problem-Solving & Debugging** – Testing, refining, and improving game performance
- **Presentation Skills** – Communicating ideas and explaining systems with confidence

The certification represents a major milestone, demonstrating that students have progressed from basic coding to **developing complete game systems with both creative and technical control**.

Scratch Level 4 Programming Course

(Year 3–4 Students)

Presented by: CIY.CLUB
Location: 3rd Floor, Education Zone, Paradise Park Shopping Center
Line: @ciyclub.paradise
Tel: 080-785-4000

Target Group: Year 3–4 students (students who have completed Scratch Level 2 or have equivalent Scratch experience)
Total Duration: 8 Weeks

Resources

- Scratch Online: https://scratch.mit.edu
- Pong (Functions + Lists): https://scratch.mit.edu/projects/1255533582
- Debugging Pong (Functions + Lists): https://scratch.mit.edu/projects/1255580082/editor/

Learning Outcomes

By the end of this course, students will be able to:

- Apply intermediate Scratch concepts including **My Blocks (functions/define), variables, lists, loops, and conditionals**
- Design and develop structured Scratch games from **planning** → **building** → **refining** → **presenting**
- Use **My Blocks** to reduce repeated code and organize programs efficiently
- Use **Lists** to manage in-game data such as player records, scores, and game history
- Explain their game's logic and structure clearly and confidently

---

Course Highlights

Structured Game Development

Students learn through real game creation—starting from analyzing and improving an existing Pong-style game, then applying the same structure to build their own project.

Computational Thinking & Problem Solving

Students practice logical thinking, problem decomposition, debugging, and code organization using functions and lists.

Project-Based Learning

Students draft, plan, build, and refine a personal game project across Weeks 6–8, gaining hands-on experience in game development.

Creativity, Collaboration & Communication

Students develop creativity and presentation skills, preparing to explain how their game works in the final showcase.

Final Course Activity

In the final week, all students will:

- Present their completed Scratch game (3–5 minutes)
- Explain how they used **functions (My Blocks)** and **lists** in their project
- Receive a **Certificate of Completion** from CIY.CLUB

Course Structure

Week	Title	Theme	Core Skills	In-Class Build	End-of-Week Deliverable
1	How Games Think: Inside the Game Brain	Game Logic & Pattern Thinking	Input → Decision → Output, reading code, spotting repeated blocks	Play + analyze a sample Pong	Students can explain game “rules” + identify repeated code
2	Build Smart Games: Using My Blocks	Functions (My Blocks)	Organizing code, reusable blocks, variables (`score`, `speed`)	Pong/Basketball Pong rebuilt using My Blocks (reset/move/hit/stop/move paddle)	Clean main loop + working Pong with speed scaling
3	Lists in Games: Saving Player History	Lists as Game Memory	`ask/answer`, lists, `length`, limiting history	Save `name + score` into `score history` (keep last 5), broadcast stop	Game remembers recent results + stops correctly
4	Game Flow & UI Screens	Start → Game → Result → Home	Broadcast system, backdrops, show/hide, state flow	Start button + Home/Result screens wired with broadcasts	Playable game with clear screen flow (no logic breaking)
5	Best Score System by Name	Lists + Index Logic	`item # of`, paired lists (`name_list`, `best_score_list`), replace-if-higher	Save best score per name (duplicate name updates only if higher)	Working “Best Score” result for each player name
6	Project Kickoff + Debug → Build	Remix, Debugging, Project Planning	Reading/fixing code, planning features, building base version	Remix + fix system, then start final project: Catch the BallOR Click the Balloon (20s)	Base version playable + plan for Week 7 improvements
7	Improve & Polish	Refinement & Readability	bug-fixing, smoother control, clear end condition, UI clarity	Upgrade gameplay feel + logic stability + simple enhancement	Stable “near-finish” version ready to present

8	Final Presentation Day	Showcase & Reflection	demo skills, explaining logic, confidence	Final testing + presentations	Each student presents their game (start → play → end → explain)

Week 1—How Games Think: Inside the Game Brain

Theme: Game Logic & Pattern Thinking

(Understanding how a game "thinks")

Lesson Objectives

By the end of this lesson, students will be able to:

- Explain that games work based on **rules and conditions**
- Identify the three core parts of a game system:
    - **Input**
    - **Decision**
    - **Output**
- Read and analyze Scratch code from a sample game
- Recognize repeated code as a problem to be solved in Level 3

Key Message (Coach Emphasis)

> Great games don't start with graphics.
> They start with **clear logic**.

Lesson Structure (45 Minutes)

Warm-up: How Does a Game Think? (5 min)

Opening Questions

- "Can a game work without code?"
- "Why does the ball in Pong bounce instead of passing through the wall?"

Student Takeaway

- Games do not act randomly
- Every action follows rules written by the programmer

Concept Talk: Inside a Game Brain (8 min)

Coach Explanation

Every game follows the same logic pattern:

1. **Input**
    - Key press
    - Collision
2. **Decision**
    - If touching → bounce
    - If score reached → win
3. **Output**
    - Movement
    - Score change

Scratch Connection

- when key pressed = Input
- if / touching = Decision
- move / change score = Output

Guided Exploration: Play & Analyze Pong (12 min)

Activity

- Students play a sample Pong game
- Coach opens and explains the code

Key Focus

- How the ball moves
- How the game detects collision
- Where the score changes

Guiding Questions

- “What happens if we remove the if block?”
- “Which blocks are repeated?”

Introduce the idea: repeated code is a problem to solve in Level 3

Concept Bridge: The Problem of Repeated Code (10 min)

Coach Demonstration

- Highlight repeated blocks such as:
    - Reset ball position
    - Increase score
    - Set starting direction

Discussion Questions

- “What if we want to change this in many places?”
- “What happens when the game becomes bigger?”

Key Idea

- Repeated code is hard to manage
- Level 3 focuses on organizing code better

Mini Challenge: Think Like a Game Developer (7 min)

Student Task (Choose One)

- Draw a simple game flow of Pong
  or
- Explain verbally:
    - “If this happens → then this happens”

Goal

- Focus on thinking, not coding
- Train logical explanation skills

Reflection & Preview (3 min)

Reflection Questions

- “What makes a game work?”
- “Which part of the code is repeated the most?”

Preview of Week 2

Next week, we will learn how to create
our own commands to solve repeated code.

Success Criteria (End of Lesson Check)

Students can:

Explain how a game works logically
Identify input, decision, and output
Point out repeated code in a game
Understand why Level 3 is different

Coach Notes (Internal – CIY)

- Do not introduce My Blocks yet
- Avoid rewriting the game from scratch
- The goal is for students to “see the game’s brain”
- If students are confused, return to:
  “What happens if there is no condition?”

Week 2–Build Smart Games: Using My Blocks

Theme: Functions (My Blocks) – Structuring a Pong Game

Lesson Objectives

By the end of this lesson, students will be able to:

- Use **My Blocks (functions)** to organize a Pong-style game
- Separate repeated game logic into clear, reusable functions
- Control the paddle using keyboard input
- Adjust game difficulty using variables (speed)
- Keep the **main game loop short and readable**

Key Message (Coach Emphasis)

> Good games are not made by long code,
> but by **clear and organized code**.

Lesson Structure (45 Minutes)

Warm-up: Why Does Pong Get Messy? (5 min)

Opening Questions

- “Where does the ball movement happen?”
- “Where do we check collision with the paddle?”
- “What happens if we want to change the ball speed?”

Coach Summary

- When logic is mixed everywhere, the game is hard to fix
- Today we organize the game using **My Blocks**

Concept Talk: My Blocks = Game Functions (7 min)

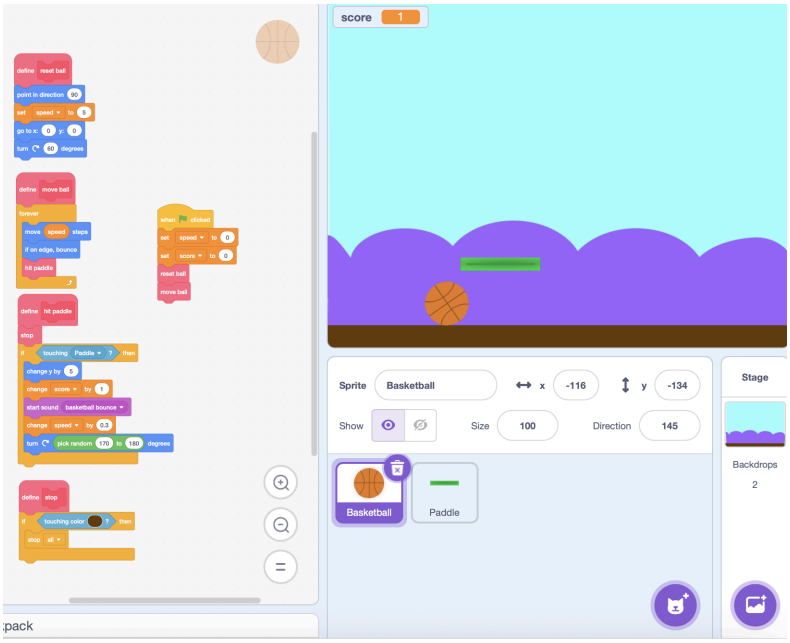
Explain Simply

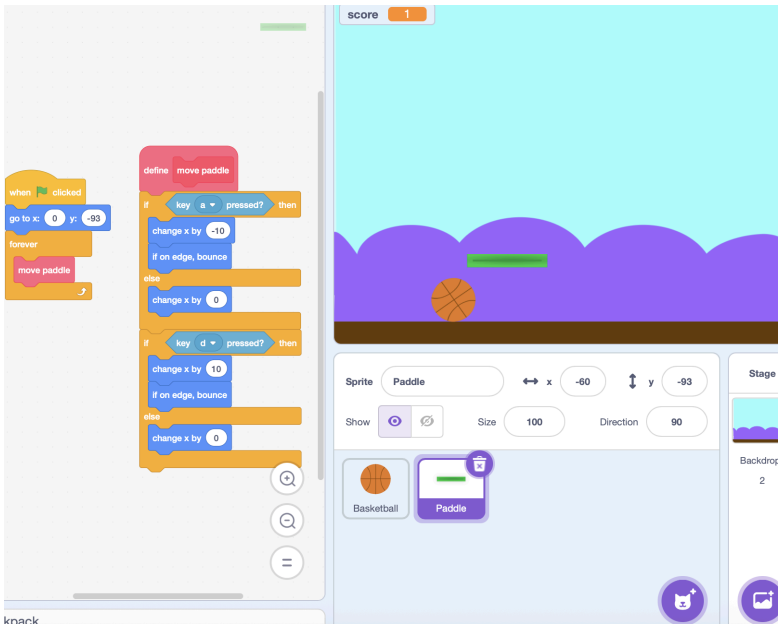
- My Blocks are **custom commands**
- Each block has **one clear job**
- We call them inside the main loop

Key Idea

- One block = one responsibility
- This makes the game easy to read and upgrade

Guided Coding: Pong Game (Based on the Project)





Sprites

- **Basketball** (Ball)
- **Paddle**

Variables

- score
- speed

My Blocks – Ball (Basketball Sprite)

A) reset ball

define reset ball

- point in direction 90
- set speed to 5
- go to x: 0 y: 0
- turn 60 degrees

Purpose

- Reset ball position and direction
- Used at the start of the game

B) move ball

define move ball

- forever
    - move (speed) steps
    - if on edge, bounce
    - call hit paddle

Purpose

- Control continuous ball movement
- Keep all movement logic in one place

C) hit paddle

define hit paddle

- if touching Paddle then
    - change y by 5
    - change score by 1
    - start sound (basketball bounce)
    - change speed by 0.3
    - turn (pick random 170 to 180) degrees

Purpose

- Handle collision with paddle
- Increase score and difficulty

D) stop

define stop

- if touching color (ground / fail color) then
    - stop all

Purpose

- End the game when the ball touches the fail area

My Blocks – Paddle (Paddle Sprite)

E) move paddle

define move paddle

- if key A pressed
    - change x by -10
    - if on edge, bounce
- else
    - change x by 0
- if key D pressed
    - change x by 10
    - if on edge, bounce
- else
    - change x by 0

Purpose

- Control paddle movement using keyboard
- Prevent paddle from leaving the screen

Main Game Scripts

Basketball Sprite

when green flag clicked

- set speed to 0
- set score to 0
- call reset ball
- call move ball

Paddle Sprite

when green flag clicked

- go to x: 0 y: -93
- forever
    - call move paddle

Mini Challenge (Optional – 5 min)

Choose one:

- Increase paddle speed
- Change how much the ball speed increases
- Adjust bounce angle range for harder gameplay

Reflection (5 min)

Reflection Questions

- “Which My Block controls the ball?”
- “Which block makes the game harder?”
- “Why is the main script short?”

Success Criteria (End of Lesson Check)

Students can:

Use **My Blocks** to separate ball and paddle logic
Control the paddle using keyboard input
Increase difficulty using the speed variable
Read and explain the main game flow

Coach Notes (Internal – CIY)

- Focus on **structure**, not decoration
- Avoid adding extra features this week
- Emphasize: *one block = one job*

- This lesson prepares students for **Week 3 (Lists / Memory)**

Week 3–Lists in Games: Saving Player History

Theme: Lists as Game Memory (Score History + Player Name)

Lesson Objectives

By the end of this lesson, students will be able to:

- Understand that a List is the game's memory
- Use ask … and wait to get the player's name
- Use answer as temporary input data
- Save player name + score into a List
- Limit the List to keep only the latest 5 records
- Use broadcast to properly stop the game

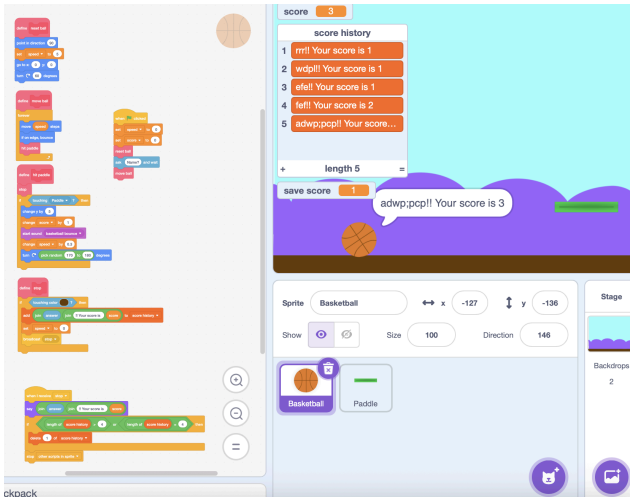
Key Message (Coach Emphasis)

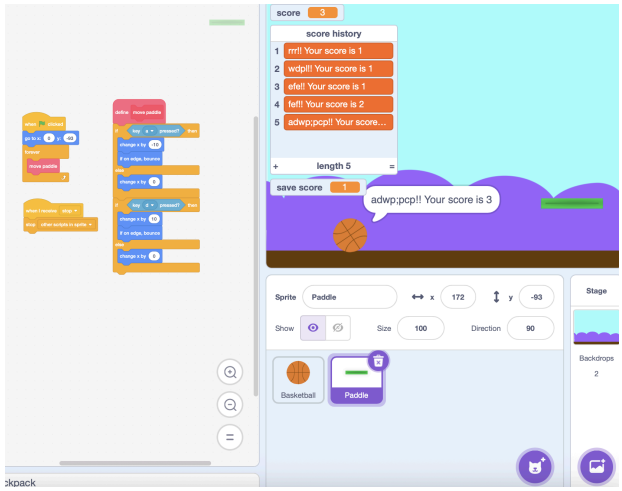
A variable remembers one value now
A list remembers many values from the past

Lesson Structure (45 Minutes)

1. Warm-up – Can games remember the past? (5 min)
2. Concept – Variable vs List + answer (8 min)
3. Build – Create score history system (22 min)
4. Mini Challenge – Improve the memory system (5 min)
5. Reflection – What did the game learn? (5 min)

Project Setup





A) Variables & List

Variables

- **score**
- **speed**
- ***(optional display)* save score**

List

- **score history**

Coach tip: Show the list on stage so students can SEE the game's memory.

B) Basketball Sprite (Ball)

Start Game + Ask Player Name

when green flag clicked

- set speed to 0
- set score to 0
- call reset ball
- ask [Name?] and wait
- call move ball

Teaching Point

- answer stores the latest text input
- It is NOT a list and is overwritten each time

My Block: reset ball

define reset ball

- point in direction 90
- set speed to 5
- go to x: 0 y: 0
- turn 60 degrees

Purpose

- Reset ball position and difficulty

My Block: move ball

define move ball

- forever
    - move (speed) steps
    - if on edge, bounce
    - call hit paddle

My Block: hit paddle

define hit paddle

- if touching Paddle then
    - change y by 5
    - change score by 1
    - start sound (basketball bounce)
    - change speed by 0.3
    - turn (pick random 170 to 180) degrees

Teaching Point

- Score and difficulty increase together

My Block: stop (Save Game Memory)

define stop

- if touching fail color (ground) then
    1. Show result
        - say (join (answer) (join [!! Your score is] (score)))
    2. Save to memory
        - add (join (answer) (join [!! Your score is] (score))) to [score history]
    3. Stop game
        - set speed to 0
        - broadcast stop

Teaching Point

- The game remembers who played and how well

C) Limit Memory to 5 Records (As in Your Project)

When game stops

when I receive [stop]

- if (length of [score history]) > 5 then
    - delete (1) of [score history]
- stop other scripts in sprite

Teaching Point

- Games do not remember everything forever
- Oldest data is removed first

D) Paddle Sprite

Start Paddle Control

when green flag clicked

- go to x: 0 y: -93
- forever
    - call move paddle

My Block: move paddle

define move paddle

- if key A pressed
    - change x by -10
    - if on edge, bounce
- else
    - change x by 0
- if key D pressed
    - change x by 10
    - if on edge, bounce
- else
    - change x by 0

Stop Paddle When Game Ends

when I receive [stop]

- stop other scripts in sprite

Success Criteria (End of Lesson)

Students can:

- Explain that a List is the game's memory
- Use ask and wait and answer
- Save name + score into score history
- Limit the list to 5 recent records
- Use broadcast to stop the game cleanly

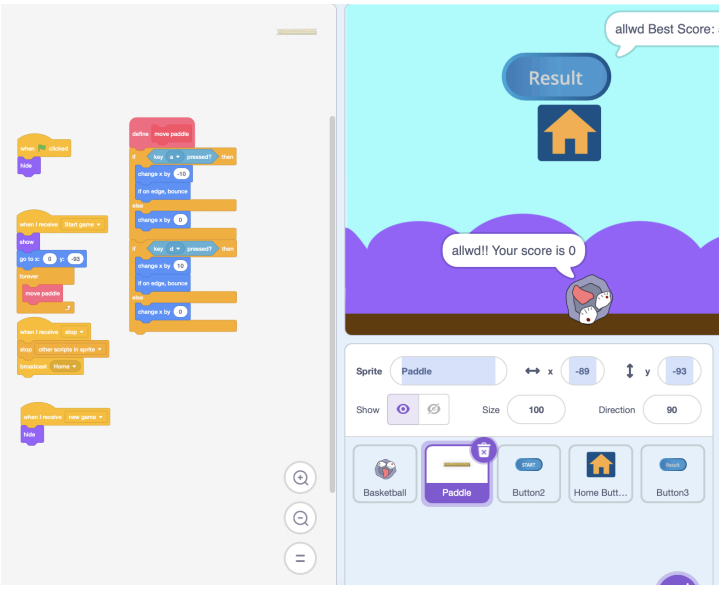
Coach Notes (CIY Internal)

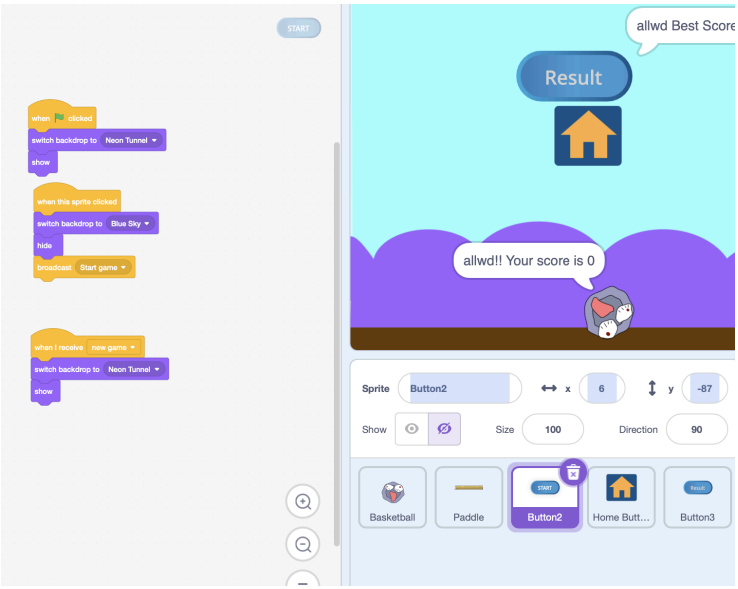
- Do NOT introduce AI or Machine Learning yet
- Use the phrase "the game remembers"
- If students are confused, return to:
    - add (name + score) to list
- Focus on understanding, not perfection

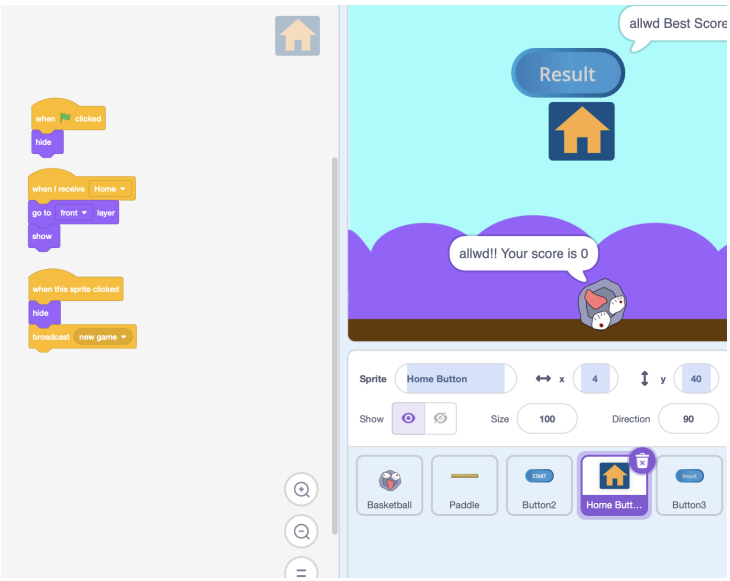
Level 3: Scratch — Week 4–5 (Coach Guide Script/Steps)

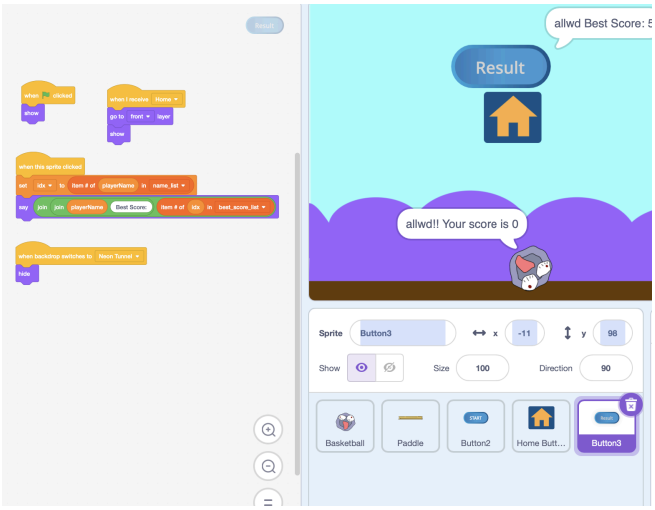
Project: **Pong/Basketball Bounce** + **Start/Home UI** + **Result Screen** + **Best Score by Name (Lists)**

`




START
when clicked
switch backdrop to Neon Tunnel
show
when this sprite clicked
switch backdrop to Blue Sky
hide
broadcast Start game
when I receive new game
switch backdrop to Neon Tunnel
show
allwd Best Score
Result
allwd!! Your score is 0
Sprite Button2
x 6
y -87
Show
Size 100
Direction 90
Basketball
Paddle
Button2
Home Butt...
Button3


when clicked
hide
when I receive Home
go to front layer
show
when this sprite clicked
hide
broadcast new game
allwd Best Score
Result
allwd!! Your score is 0
Sprite Home Button
x 4
y 40
Show
Size 100
Direction 90
Basketball
Paddle
Button2
Home Butt...
Button3



Week 4 — Game Flow + UI Screens (Start → Game → Result → Home)

Learning goals (tell students)

- Make a game feel “real” with **screens**: Home / Playing / Result
- Control screens using **broadcast messages**
- Use **show/hide**, **switch backdrop**, **stop other scripts** correctly

A) Prep (2–3 min)

Sprites you should have

- Basketball (the ball)
- Paddle (player control)
- Button2 (Start button)
- Home Button (go back to home)
- Button3 (Result button / show result info)

Backdrops

- Neon Tunnel (Home/Result screen)
- Blue Sky (Game screen)

Broadcast names (exact, keep consistent)

- Start game

- stop
- Home
- new game

Coach note: If students used different names, **choose one set and rename all** so it matches everywhere.

B) Screen logic (Home / Start)

1) Stage (or Start button sprite) — show Home screen on green flag

When green flag clicked

- switch backdrop to [Neon Tunnel]
- broadcast [Home] (optional, but clean)
- Make sure:
    - Start button is visible
    - Result button hidden
    - Home button hidden
    - Ball/paddle hidden

Coach script

> "Green flag = reset the project to the Home screen."

C) Start Button (Button2) — start playing

2) Button2 (Start)

When green flag clicked

- show
- go to front layer

When this sprite clicked

- switch backdrop to [Blue Sky]
- hide (hide start button)
- broadcast [Start game]

Coach script

> "When we press Start, we switch to the game backdrop and tell the game to begin."

D) Game Sprites — respond to Start game

3) Paddle — appear and enable movement

When I receive [Start game]

- show
- go to x: 0 y: -93 (or your chosen position)
- Start movement loop (two good patterns)

Pattern A (My Block style)

- forever
    - move paddle (My Block)

Pattern B (Direct if keys)

- forever
    - if key (a) pressed → change x by (-10)
    - if key (d) pressed → change x by (10)
    - add edge protection (see below)

Edge protection (recommended)
Instead of if on edge, bounce (which can feel weird), use:

- set x to (max(min(x, 220), -220)) (advanced)
  OR simpler:
- if x > 220 set x to 220
- if x < -220 set x to -220

Coach note: If you want to keep it simple, if on edge, bounce is okay for Week 4.

4) Basketball — appear and start moving

When I receive [Start game]

- Reset variables:
    - set [score v] to 0
    - set [speed v] to 5 (or your value)
- Then:
    - call reset ball
    - call move ball

My Blocks (recommended)

- reset ball

        - go to x: 0 y: 0
        - point in direction 90
        - turn (pick random 30 to 120) degrees (or your “turn 60 degrees”)
- move ball
    - forever
        - move (speed) steps
        - if on edge, bounce
        - call hit paddle
        - call stop check (optional)

E) Ending the game → Result screen

5) Basketball — detect game over

Decide your “fail condition”

- Touching ground color
- y position below a line (e.g., y < -150)
- Touching a “red line” sprite

When fail happens

- broadcast [stop]

6) Stop message behavior

Paddle

- when I receive [stop]
    - stop other scripts in sprite
    - (optional) hide

Basketball

- when I receive [stop]
    - stop other scripts in sprite
    - (optional) hide
    - show result UI:
        - broadcast [new game] OR broadcast [Home] depends on your flow

Coach script

> “Game over = stop movement loops, then show Result/Home buttons.”

F) Result button + Home button

7) Button3 (Result)

When I receive [stop]

- show
- go to front layer

When this sprite clicked

- Show score message / result panel
- (Week 5 will improve this into “name + best score”)

8) Home Button

When I receive [stop]

- show
- go to front layer

When this sprite clicked

- broadcast [Home]

When I receive [Home]

- switch backdrop to [Neon Tunnel] (Stage or button can do it)
- Hide in-game sprites:
    - ball, paddle
- Show start button again:
    - broadcast [new game] OR directly show Button2
- Hide result/home buttons if needed

Success check (end of Week 4)

- Start screen shows Start button
- Clicking Start begins play
- Game ends and stops movement
- Result/Home buttons appear
- Home returns to start screen cleanly

Week 5 — Save Name + Best Score (Lists + Replace if Higher)

Learning goals (tell students)

- Use **Lists** to store data
- Understand **index** (position in list)
- Save “best score per player name”
- If name repeats → keep **highest** score only

A) Create Lists + Variables

Create 2 lists:

- name_list
- best_score_list

Create variables (recommended):

- playerName
- idx
- oldBest

Coach note: These two lists must always match by index:

- name_list[1] goes with best_score_list[1]

B) Ask for name (best place: when starting game)

In Basketball or Stage, on Start game:

When I receive [Start game]

- ask [Name?] and wait
- set [playerName v] to (answer)

(Optional cleanup)

- If empty name:
    - if <(playerName) = []> then set playerName to [Player]

Coach script:

> “We store the name first, then start scoring.”

C) Save best score (My Block)

Create a My Block in Basketball:

save best score

Logic

1. Find index:
- set [idx v] to (item # of (playerName) in [name_list])
2. If not found (idx = 0):
- add (playerName) to [name_list]
- add (score) to [best_score_list]
3. Else (name exists):
- set [oldBest v] to (item (idx) of [best_score_list])
- if <(score) > (oldBest)> then
    - replace item (idx) of [best_score_list] with (score)

Coach script

> "item # tells us where the name is. If it's 0, it's new. If it's not 0, we compare and replace only if higher."

D) Call save logic at game over

In Basketball, where you detect fail:

- broadcast [stop]
- call save best score

Order tip:

- Save before stopping everything is safer, but either can work if you don't stop the script too early.

Good pattern:

- On fail:
    - call save best score
    - broadcast [stop]

E) Show the best score on Result screen (Button3)

In Button3 (Result button):

When this sprite clicked

- set [idx v] to (item # of (playerName) in [name_list])
- if <(idx) > 0> then
    - say (join (playerName) (join [Best Score:] (item (idx) of [best_score_list]))) for 2 seconds
    - (optional) also show current score:
        - say (join [Your score:] (score))

Coach note:

- If they click Result but playerName isn't set, make sure name is asked earlier (Start game) or ask again on Result.

F) Optional: "Top 1 Best Player" (max of list)

If you want "highest score overall + name":

- Use a loop to scan best_score_list and find max + index
  (This is a great Week 5 extension, not required.)

Week 5 Success criteria

Students can:

- Save name + score to two lists
- If name repeats, keep only the **highest** score
- Result screen shows **playerName + best score** correctly
- Home/Start flow still works without breaking

WEEK 6 – Project Kickoff, Debugging & Planning

Scratch Trainee – Final Mini Project Begins

Big Picture for Students (Coach MUST say this first)

> "From today until Week 8,
> you will work on **ONE game project**.
> You will improve it every week
> and present it in the final class."

Explain clearly:

- This is **not a one-day project**
- What they start today → will be shown in Week 8
- Today = understand, fix, and plan

Final Project Choices (Show on Screen)

Students choose **ONE** project only:

Option A: Catch the Ball (Recommended)

- Ball falls from the top
- Paddle catches the ball
- +1 score per catch
- Miss = lose / end game

Option B: Click the Balloon (Timed Game)

- Balloon appears randomly
- Click to get points
- 20-second timer
- Save final score at the end

Coach note:
👉 Tell kids **this is enough** — no extra features yet

Session Objective (Student-Friendly)

By the end of Week 6, students will:

- Understand how a full Scratch game is structured
- Debug an existing project correctly

- Choose a final project and start building the base version

Part 1: Project Brief & Mindset (10 minutes)

Coach Talk (Very Important)

Say slowly and clearly:

> "Good programmers do not start by coding.
> They start by **reading and fixing** code."

Explain:

- Debugging = learning from mistakes
- Remixing = professional skill
- Today is about **understanding**, not speed

Part 2: Debugging Before Building (25–30 minutes)

Student Task

1. Open the remix project
   https://scratch.mit.edu/projects/1255580082/editor/
2. Click **Remix**
3. Run the game fully:
   - Start
   - Play
   - End
   - Back to Home

Debug Focus Areas (Coach-Guided)

Game Flow

Students check:

- Start button works
- Game actually begins
- Home button resets game

Coach prompts:

- "Which broadcast starts the game?"
- "What should hide when the game starts?"
- "What should show when the game ends?"

Movement & Control

Students check:

- Paddle moves smoothly
- Ball resets correctly
- No sprite moves before game starts

Coach prompts:

- “Which script runs forever?”
- “Should this run before the start of the game?”

Score Logic

Students check:

- Score starts at 0
- Score increases correctly
- Score stops after game over

Coach explanation:

> “Score logic must follow the game state.
> No game state = no score change.”

Debug Rule

Students must fix **at least ONE issue** and explain:

- What was wrong
- What they changed
- Why it works now

Part 3: Final Project Selection & Planning (10 minutes)

Student Choice

Each student chooses:

- Catch the Ball **OR**
- Click the Balloon

Coach checks:

- One choice only

- No mixing projects

Project Planning (Very Simple)

Students answer verbally or on paper:

1. What sprite gives a score?
2. How does the game start?
3. How does the game end?

Part 4: Start Building Base Version (15–20 minutes)

Allowed Scope (Week 6 ONLY)

Students build:

- Start button
- Basic gameplay
- Score works
- End condition works

No leaderboard
No best score yet
No polish

Coach reminder:

> “Base version first.
> Fancy version later.”

End of Class Wrap-Up (5 minutes)

Ask students:

- “What project did you choose?”
- “What part did you fix today?”
- “What will you improve next week?”

Tell them clearly:

> “Next week, we will improve this game, not start a new one.”

Coach Internal Notes (CIY)

- Week 6 = **Foundation**
- Do NOT rush kids to finish
- Debugging is more important than features
- If a student finishes early:
    - Improve movement
    - Improve visuals
    - Add simple sound

Week 7–8 – Final Project Development & Presentation

Target Group: Year 3–4
Project Type: Mini Game (Catch the Ball OR Click the Balloon)
Outcome: Playable game + student presentation

Overview for Coaches (Big Picture)

- Students **must continue the same project** started in Week 6
- No new games are allowed
- Focus shifts from “building” → “improving” → “presenting”
- By Week 8, every student should be able to:
    - Play their game
    - Explain how it works
    - Show confidence in their work

WEEK 7 – Improve, Polish & Prepare

Week 7 Objectives (Tell Students)

> “Today we make our game better, smoother, and clearer.”

By the end of Week 7, students will:

- Improve gameplay quality
- Fix bugs and logic issues
- Prepare their game for final presentation

Part 1: Recap & Direction (5 minutes)

Coach asks each student:

- What is your game called?
- How does your game start?
- How do you get score?
- How does your game end?

Coach reminder:

> “You are improving your game, not starting a new one.”

Part 2: Mandatory Improvements (20–25 minutes)

Every student must complete **ALL 4 areas** below.

Smooth Gameplay

Choose **at least one**:

- Reduce movement speed for better control
- Use glide instead of go to
- Add small delay (e.g. wait 0.05 seconds)
- Improve collision response (no double counting)

Coach question:

- “Does your game feel smooth or too fast?”

Clear Game Flow

The game must clearly show:

- Start screen
- Playing state
- Game over screen

Students must correctly use:

- broadcast
- show / hide
- switch backdrop

Coach question:

- “What state is your game in right now?”

Correct Score Logic

Check carefully:

- Score starts at 0
- Score increases only during gameplay
- Score stops changing after game over

Common mistakes to fix:

- Score increases before Start
- Score continues after game ends

Clear End Condition

Choose **one**:

- Timer reaches 0
- Player misses the ball
- Player touches an obstacle
- Score reaches a target

Coach reminder:

> "Every game must know when it is finished."

Part 3: Optional Enhancements (Fast Finishers)

Students may choose **ONLY ONE**:

- Add sound effects
- Change background or sprites
- Increase difficulty over time
- Add simple animation

Coach rule:

> "One improvement is enough. Do not overbuild."

Part 4: Presentation Preparation (10 minutes)

Students prepare answers to **3 simple questions**:

1. What is your game called?
2. How do you play it?
3. How do you get a score?

Coach note:

- No memorizing
- Speak naturally

End of Week 7 – Coach Wrap-Up

Coach says clearly:

> “Next week is presentation week.
> No big changes after today.”

WEEK 8 – Final Presentation & Reflection

Week 8 Objectives (Tell Students)

> “Today you show your game and explain how it works.”

By the end of Week 8, students will:

- Present their game confidently
- Explain game logic in simple words
- Reflect on what they learned

Part 1: Final Game Check (10 minutes)

Students check:

- Start button works
- Game plays correctly
- Game ends properly
- Home / restart works
- Score resets correctly

Allowed fixes:

- Small bugs
- UI issues
- Score reset problems

Not allowed:

- New features
- Major redesigns

Part 2: Presentation Time (25–30 minutes)

Presentation Format (≈2 minutes per student)

Each student:

1. Shows Start screen
2. Clicks Start
3. Plays the game briefly
4. Ends the game
5. Explains how it works

Suggested Student Script (Optional)

> "This is my game.
> You start by clicking Start.
> You get a score by _____.
> The game ends when _____."

Part 3: Coach Feedback (Live & Positive)

For each student, coach gives:

- One positive observation
- One improvement from Week 6 → Week 8

Rules:

- No comparison between students
- Focus on effort and progress

Part 4: Reflection (5 minutes)

Coach asks:

- What was the hardest part?
- What are you most proud of?

Coach closes with:

> "You didn't just play a game.
> You built one."

Final Learning Outcomes (CIY Internal)

After Week 8, students can:

- Build a complete Scratch game
- Debug basic logic problems
- Use variables, broadcasts, and conditions
- Present and explain their own project

LEVEL 5: Scratch Advanced Game Systems

Game Architecture & System Integration
Duration: 8 Weeks
Presented by: CIY.CLUB

Learning Outcomes

By the end of this course, students will be able to:

- Design and develop **advanced game systems** with multiple interconnected mechanics
- Apply high-level programming concepts including **clones, lists, game states, UI systems, and event-driven architecture**
- Structure games using **Start → Play → End flow with multi-level progression**
- Implement **enemy systems, physics-based movement, and dynamic difficulty scaling**
- Manage complex data using **variables and lists** (score, health, states, progression)
- Organize projects using **broadcast systems and modular logic design**
- Plan, build, and refine a **complete game independently**
- Debug and optimize systems for improved gameplay and performance
- Present their project with clear explanation of **game systems and logic flow**

Course Highlights

Advanced Game Architecture
Students move beyond creating games to understanding how games function as **integrated systems**.

System Integration Thinking
Students learn how multiple systems connect and interact:
Player → Enemy → Physics → Score → UI → Level Progression

Real Game Mechanics
Projects include realistic mechanics such as **gravity, enemy spawning, health systems, and difficulty scaling**.

System-Level Project-Based Learning
Each lesson builds a system that contributes to a complete final game.

Independent Development with Coaching
Students take ownership of their projects while receiving structured guidance from coaches.

Final Game Showcase
Students present a complete game system and explain how each component works together.

Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Game System Review	Game structure (Start → Play → End), events, variables	Analysis + Guided Practice	Students understand how a complete game system is structured
2	Clone System	Dynamic object creation (create/delete clone)	Guided Project	Students create multiple objects using clone systems
3	Collision & Interaction	Interaction logic and event responses	Guided Project	Students implement collision-based interactions
4	Physics Movement System	Gravity, velocity, jumping mechanics	Guided Project	Students create smooth and responsive movement systems
5	Level & Difficulty System	Progression and difficulty scaling	Guided → Semi-guide d	Students design games with increasing challenge
6	Game UI & Data System	Health, timer, score, lists	Semi-guide d	Students implement UI systems and manage game data

7	Final Project Development	System integration and debugging	Independent Build	Students develop a complete game combining all systems
8	Final Showcase & Presentation	Communication and system explanation	Presentation	Students present and explain their full game system

The Final Week (Week 8) Will Include

- **Final Game System Showcase**
  Students present their completed game to peers, parents, and coaches
- **System Architecture Explanation**
  Students explain how different systems work together (game flow, UI, mechanics, logic)
- **Reflection Activity**
  Students reflect on their learning journey and problem-solving process
- **Celebration Activities**
  Activities to recognize achievement and build confidence

Certification & Recognition

All students who successfully complete the **Level 5: Scratch Advanced Game Systems Program** will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Advanced Game Development** – Designing and building complete game systems
- **System Thinking & Architecture** – Understanding how multiple systems connect and interact
- **Advanced Coding Concepts** – Applying clones, lists, variables, and event-driven logic
- **Problem-Solving & Debugging** – Managing and improving complex systems
- **Independent Project Development** – Building a full game from concept to completion
- **Presentation Skills** – Explaining system logic and design with clarity

The certification represents a high-level milestone, demonstrating that students have progressed from basic coding to **advanced system design and game architecture thinking**.

AI Explorer

LEVEL 6: AI Explorer

AI Thinking & Creative Communication
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Understand what AI is and how it works (Input → Data → Pattern → Output)
- Distinguish between **AI and human thinking**
- Ask clear and structured questions (prompting) to guide AI effectively
- Use AI as a **support tool**, not a replacement for thinking
- Identify risks and apply **safe and responsible AI usage**
- Evaluate AI-generated content (images, text) with critical thinking
- Create simple presentations using AI-supported tools (GPT + Gamma)
- Communicate ideas confidently using their **own words**
- Plan and present a **personal project with AI support**

🔹 Course Highlights

AI Thinking Foundation
Students learn how AI works and develop the mindset to use it correctly.

Prompting & Communication Skills
Students learn how to ask better questions to get better results.

Responsible AI Use
Strong focus on safety, ethics, and understanding AI limitations.

Creative AI Production
Students use AI tools to support storytelling, visuals, and presentations.

Confidence & Communication
Students practice speaking, presenting, and expressing their ideas clearly.

Project-Based Learning
Students create and present their own project using AI as a support tool.

🔹 Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	What Is AI & How AI Works	AI vs Human / Input → Output	Discussion + Activity	Students explain what AI is and how it works
2	Prompting & Asking Questions	Good vs Bad prompts	Guided Practice	Students create clear prompts
3	AI Safety & Responsibility	Safe use / Not sharing info	Discussion + Scenario	Students use AI safely and responsibly
4	Images & Writing with AI	Real vs AI / Support writing	Demo + Practice	Students question AI outputs
5	Slides & Presentation with AI	GPT + Gamma	Guided Creation	Students create simple slides
6	Project Planning & Build	Topic selection / structure	Planning + Build	Students create project draft
7	Practice & Refinement	Speaking / clarity	Practice + Feedback	Students improve presentation
8	Final Presentation	Communication / reflection	Presentation	Students present project confidently

---

🔹 The Final Week (Week 8) Will Include

- **Final AI Project Presentation**
  Students present their project to peers, parents, and coaches
- **AI Usage Explanation**
  Students explain what AI helped with and what they created themselves
- **Reflection Activity**
  Students reflect on how they used AI and what they learned

- **Celebration Activities**
  Group activities to build confidence and celebrate achievements

---

🔹 Certification & Recognition

All students who successfully complete the **Level 6: AI Explorer Program** will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **AI Understanding** – Knowing what AI is and how it works
- **Prompting & Communication** – Asking clear and meaningful questions
- **Responsible AI Use** – Using AI safely and ethically
- **Critical Thinking** – Evaluating AI-generated content
- **Creative AI Application** – Using AI to support ideas and presentations
- **Presentation Skills** – Communicating ideas with confidence

The certification represents a key milestone, showing that students have progressed from **coding systems to thinking with AI systems responsibly and creatively**.

Scratch and AI

LEVEL 7: AI to Scratch System Builder

AI Planning → System Design → Scratch Implementation
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Translate AI-generated ideas into **structured Scratch systems**
- Design logic using **flowcharts before coding**
- Apply system thinking: **Input** → **Process** → **Output** → **Feedback**
- Build complete game flow: **Start** → **Play** → **End**
- Use AI as a **planning assistant**, not a replacement
- Develop a project from **idea** → **plan** → **build** → **refine** → **present**
- Debug systems based on logic understanding
- Explain and defend their system using **flowcharts and reasoning**

🔹 Course Highlights

AI → System Thinking Bridge
Students transform ideas into structured systems before coding.

Flowchart-Based Learning
Strong emphasis on planning logic visually before implementation.

Early Project Start (Week 5)
Students begin their main project earlier, allowing deeper development.

System-Level Thinking
Focus on how systems connect:
Input → Logic → Output → Feedback

Presentation & Defense Skills
Students explain and answer questions about their logic confidently.

Challenge-Based Final Showcase
Final presentation structured like a mini competition.

🔹 Course Structure (Updated)

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Review & System Thinking	Scratch + AI recap / system flow	Discussion + Analysis	Students understand full game system structure
2	Flowchart Basics	Decision flow / logic mapping	Guided Practice	Students create simple flowcharts
3	AI → Flowchart	Idea → structured logic	Guided Activity	Students convert ideas into flowcharts
4	Flowchart → Scratch	Mapping plan to code	Guided Project	Students build small system from flowchart
5	🚀 Project Kickoff	Project planning + system design	Planning + Start Build	Students create flowchart + start project
6	Project Development	Build system	Build + Coaching	Students develop working core system
7	Refinement + Presentation Practice	Debugging + explanation	Build + Practice	Students refine system + explain logic
8	Final Challenge & Showcase	Competition + presentation	Presentation	Students present and defend project

---

🔹 The Final Week (Week 8) Will Include

- **Final Project Challenge (Competition Style)**
  Students present their project in a structured format
- **System Explanation & Logic Defense**
  Students explain flowchart, system logic, and decisions
- **Q&A Session**
  Coaches ask questions to test understanding

- **Peer Learning & Feedback**
  Students observe and learn from each other
- **Celebration Activities**
  Build confidence and recognize achievements

🔹 Certification & Recognition

All students who successfully complete the
Level 7: AI to Scratch System Builder Program
will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **System Design Thinking** – Planning using flowcharts
- **AI-Assisted Planning** – Using AI for structured idea development
- **Scratch System Implementation** – Building from plan to working system
- **Problem-Solving & Debugging** – Refining systems logically
- **Project Development** – Creating a complete system independently
- **Presentation & Defense Skills** – Explaining and answering questions clearly

The certification represents a major milestone, showing that students have progressed from:

👉 **Coding → Thinking → System Design → Real Implementation**

Python

Python LEVEL 1(Logic First – Foundation Level)

Python Core Fundamentals (Scratch to Text-Based Coding)
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Transition from **Scratch (block-based)** to **Python (text-based coding)**
- Use **print()** to display outputs clearly
- Understand **basic data types**: int, float, str
- Use **variables** to store and manage data
- Use **input()** to receive user data
- Perform **basic calculations** (+, -, *, /)
- Combine input, variables, and calculations into simple programs
- Apply **Input → Process → Output thinking**
- Build confidence in writing and explaining simple Python programs

---

🔹 Course Highlights

Scratch → Python Transition
Students connect visual logic (Scratch) to real coding syntax step by step.

Core Logic First
Focus on understanding how programs work before adding complexity.

Hands-On Mini Programs
Students create meaningful programs such as:

- Name generator
- Personal info system
- Basic calculator

Selective Typing Practice
Typing is integrated **only in key sessions** to support coding (not every class).

Confidence & Communication
Students explain their logic and present their work clearly.

🔹 Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Scratch → Python Bridge	Block vs Text / print()	Discussion + Practice	Students understand Python vs Scratch and use print()
2	print() & Output	Display messages	Guided Practice + (Typing Intro)	Students create output correctly
3	Variables & Data Types	int / float / str	Practice	Students store and display data
4	Input System	input() / interaction	Guided Practice + (Typing Practice)	Students create input-based programs
5	Basic Calculation	+, -, *, /	Guided Practice	Students perform calculations correctly
6	Input → Process → Output	System thinking	Guided Project + (Typing Practice)	Students build a simple calculator
7	Mini Project	Combine concepts	Build + Coaching	Students create a working mini system
8	Final Project + Presentation	Logic explanation	Presentation	Students present and explain their program

---

🔹 The Final Week (Week 8) Will Include

- **Final Project Presentation**
  Students present their Python mini project

- **Logic Explanation**
  Students explain:
  👉 Input → Process → Output
- **Reflection Activity**
  Students reflect on what they learned and improved
- **Celebration Activities**
  Activities to build confidence and engagement

🔹 Certification & Recognition

All students who successfully complete the
Python Basics Program (CIY.CLUB)
will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Python Foundations** – Using print(), variables, input, and calculations
- **Logical Thinking** – Understanding Input → Process → Output
- **Problem-Solving Skills** – Building simple working programs
- **Scratch to Python Transition** – Moving from visual to text-based coding
- **Communication Skills** – Explaining program logic clearly

The certification represents a foundational milestone, showing that students have progressed from:

👉 **Visual Coding → Basic Logic → Text-Based Programming**

Python LEVEL 2: Logic & Decision Systems

Condition • Comparison • Control Flow (From Logic → Decision Systems)
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Use **comparison operators** (==, >, <, >=, <=) to evaluate conditions
- Apply **if / elif / else** to control program decisions
- Build **nested if/else systems** for more complex logic
- Combine **input** → **condition** → **output** into decision-based programs
- Understand how programs make decisions like real systems
- Debug logical errors and improve decision flow
- Develop structured thinking through **control flow design**
- Create interactive programs such as **grade checker, quiz system, decision games**
- Present and explain their logic clearly using step-by-step reasoning

---

🔹 Course Highlights

From Logic → Decision Making
Students move from basic programming to systems that can “make decisions”.

Comparison & Control Flow Mastery
Focus on how programs evaluate conditions and respond accordingly.

Real-World Systems
Students build programs such as:

- Grade checker
- Access system (pass/fail)
- Quiz system

Nested Thinking (Advanced Logic)
Students learn to handle multiple conditions and deeper logic structures.

Project-Based Learning
Students apply all concepts into interactive mini systems and projects.

Selective Typing Practice
Typing is included only when needed to support coding fluency.

🔹 Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Review + Comparison Operators	==, >, <, >=, <=	Practice + Mini Tasks	Students compare values correctly
2	if Statement	Basic decision making	Guided Practice	Students use if to control output
3	if / else	Two-way decisions	Guided Practice + (Typing)	Students build pass/fail systems
4	if / elif / else	Multi-condition logic	Guided Practice	Students handle multiple conditions
5	Nested if/else	Complex decision systems	Guided Project	Students build nested logic systems
6	Decision System Project	Input → Logic → Output	Build + Coaching	Students create a working system
7	Project Development	Debugging + refinement	Build + Feedback	Students improve and explain logic
8	Final Project + Presentation	Logic explanation	Presentation	Students present and defend their system

---

🔹 The Final Week (Week 8) Will Include

- **Final Project Presentation**
  Students present their decision-based program

- **Logic Explanation**
  Students explain:
  👉 Input → Condition → Output
- **Q&A Session**
  Coaches ask questions to test understanding
- **Reflection Activity**
  Students reflect on how they improved their logic
- **Celebration Activities**
  Activities to build confidence and engagement

🔹 Certification & Recognition

All students who successfully complete the
Python Level 2: Logic & Decision Systems Program (CIY.CLUB)
will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Control Flow Logic** – Using if, elif, else, and nested conditions
- **Comparison & Evaluation** – Applying logical conditions correctly
- **System Thinking** – Understanding Input → Condition → Output
- **Problem-Solving Skills** – Building structured decision systems
- **Debugging Skills** – Identifying and fixing logic errors
- **Communication Skills** – Explaining logic clearly and confidently

The certification represents a key progression, showing that students have advanced from:

👉 **Basic Programming → Logical Decision Making → System Thinking**

Python LEVEL 3: System & Function Builder

Functions • Reusability • System Flow (From Decision → Structured Systems)
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Understand the purpose of **functions (def)** and why they are important
- Break programs into **smaller reusable parts (modular thinking)**
- Design structured systems using **functions + conditions + loops**
- Understand the concept of **while True loop (continuous system)**
- Apply **Input** → **Process** → **Output** → **Repeat systems**
- Plan and design programs using **story-based logic flow**
- Build interactive programs with structured function design
- Debug and improve program structure for clarity and efficiency
- Present and explain system logic clearly

🔹 Course Highlights

Function = System Thinking
Students learn that functions are not just code, but “systems inside systems”.

Modular Programming
Students break large problems into smaller manageable parts.

While Loop Introduction
Students understand how systems can run continuously (e.g., game loop).

Story-Based Programming
Students design programs like stories:
Start → Choice → Outcome → Repeat

Early Project Design (Week 4)
Students begin planning their own project early to build deeper understanding.

Independent System Building
Students move toward building structured programs independently.

🔹 Course Structure

Week	Theme	Key Concept	Activity Format	Success Criteria (Learning Outcome)
1	Introduction to Functions	def / why functions matter	Guided Practice	Students create simple reusable functions
2	Functions + Parameters	Passing values / reuse logic	Guided Practice	Students use functions with inputs
3	System Flow with Functions	Function + condition	Guided Mini Project	Students structure programs using functions
4	🚀 Project Design (Story-Based)	Story flow / system design	Planning + Coaching	Students design their own program (story/system)
5	While Loop (Intro)	while True / continuous system	Guided Practice	Students understand loop-based systems
6	Project Build (Part 1)	Function + loop integration	Build + Coaching	Students implement core system
7	Project Build (Part 2)	Debugging + refinement	Build + Feedback	Students refine and structure their program
8	Final Project + Presentation	System explanation	Presentation	Students present and explain full system

---

🔹 The Final Week (Week 8) Will Include

- **Final Project Presentation**
  Students present their program (story/system)
- **System Logic Explanation**
  Students explain:
  👉 Function → Flow → Loop → Output
- **Q&A Session**
  Coaches ask questions to test system understanding

- **Reflection Activity**
  Students reflect on how they structured their program
- **Celebration Activities**
  Build confidence and engagement

🔹 Certification & Recognition

All students who successfully complete the
Python Level 3: System & Function Builder Program (CIY.CLUB)
will receive a **Certificate of Completion from CIY.CLUB**.

This certificate recognizes students' achievements in:

- **Function & Modular Thinking** – Breaking programs into reusable systems
- **System Design** – Structuring programs with logic and flow
- **Loop Understanding** – Applying continuous systems (while True)
- **Problem-Solving & Debugging** – Improving program structure and logic
- **Project Development** – Designing and building a complete system
- **Communication Skills** – Explaining system logic clearly

The certification represents a major progression, showing that students have advanced from:

👉 **Decision-Based Coding → Structured System Design → Modular Programming**

Python LEVEL 4: AI System & Intelligence Builder

(ต่อจาก Level 3: System Builder → Level 4: Make System Intelligent)
Duration: 8 Weeks
Presented by: CIY.CLUB

🔹 Learning Outcomes

By the end of this course, students will be able to:

- Understand the difference between rule-based systems and AI systems
- Apply **Data** → **Pattern** → **Prediction** → **Decision** as a core AI model
- Use lists and variables as memory systems in programs
- Build programs that adapt behavior based on past data
- Identify patterns in data and use them for simple prediction
- Design AI-like systems that change behavior dynamically
- Evaluate AI outputs and understand limitations and errors
- Apply basic AI ethics and responsible thinking
- Present and explain AI system logic clearly

🔹 Course Highlights

From System → Intelligent System

Students upgrade programs from fixed logic to adaptive behavior using data.

Data-Driven Thinking

Students learn that AI decisions come from data, not randomness.

AI Memory Systems

Programs store past information and use it to improve decisions.

Pattern & Prediction

Students explore how AI identifies patterns and predicts outcomes.

Explainable AI

Students learn to explain **why AI makes decisions**, not just what it outputs.

AI Project-Based Learning

Students build their own AI-style systems and test real behavior.

🔹 Course Structure (8 Weeks)

Week	Theme	Key Concept	Activity Format	Success Criteria
1	Logic Review & AI Readiness	Logic → AI foundation	Guided Practice	Understand that AI starts from logic
2	Data & Decision	Lists / data-driven logic	Guided Practice	Use data to influence decisions
3	Introduction to AI	Rule vs AI system	Concept + Practice	Explain how AI works
4	Decision-Making AI	Decision trees / priority	Mini Project	Build AI decision system
5	AI Memory	Data storage / behavior change	Guided Practice	Use memory in programs
6	Prediction & Patterns	Pattern recognition	Guided Practice	Build simple prediction system
7	AI Project Build	System integration	Build + Coaching	Create AI-based system
8	AI Showcase & Ethics	Presentation + reflection	Presentation	Explain full AI logic clearly

---

🔹 The Final Week (Week 8) Will Include

AI Project Presentation

Students present their AI-based program (game / advisor / system)

AI Logic Explanation

Students explain:
👉 Data → Pattern → Prediction → Decision

Q&A Session

Coaches test understanding of AI logic and reasoning

AI Ethics Reflection

Students reflect on:

- AI limitations
- Data quality
- Responsible use

Celebration Activities

Build confidence and ownership of AI learning

🔹 Certification & Recognition

All students who successfully complete the
Python Level 4: AI System & Intelligence Builder Program (CIY.CLUB)
will receive a Certificate of Completion.

This certification recognizes students' development in:

- **AI Thinking & System Intelligence**
  Understanding how systems use data to make decisions
- **Data & Memory Systems**
  Using data storage to influence behavior
- **Pattern Recognition & Prediction**
  Identifying trends and predicting outcomes
- **Adaptive System Design**
  Building systems that change behavior dynamically
- **Problem-Solving & Debugging**
  Evaluating and improving AI logic

- **Ethical Awareness**
  Understanding limitations and responsible AI use
- **Communication Skills**
  Explaining AI logic clearly and confidently

Python LEVEL 5: Turtle Graphics & Visual Systems

CIY.CLUB Thailand — Structured Creative Coding Program

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Understand how visual systems are built using Python turtle graphics
- Apply loops and functions to create structured drawing systems
- Use coordinates and positioning to control object placement on screen
- Design and build simple animation and game logic (Turtle Race)
- Develop logical thinking through visual feedback and iteration
- Debug and improve their own code through testing and refinement
- Present their work and explain the logic behind their program

---

🚀 Course Highlights

- **Visual Coding Approach**
  Students learn programming through immediate visual feedback, making abstract concepts easier to understand.
- **System Thinking Foundation**
  Every drawing is treated as a system: Input → Process → Output (Visual Result)
- **Creative + Technical Balance**
  Students don't just code — they design patterns, scenes, and animations.
- **Progressive Skill Building**
  From simple movement → loops → functions → full mini-game system
- **Project-Based Learning**
  Each phase builds toward a final creative project and presentation

---

📅 Course Structure (8 Weeks)

Week	Theme	Key Concepts	Activity Format	Success Criteria

1	Introduction to Turtle	turtle module, movement commands, pen control	Draw basic shapes (square, triangle)	Student can control turtle and create simple shapes
2	Drawing with Loops	for loops, repetition, angles	Create geometric shapes using loops	Student uses loops to reduce repetition
3	Functions & Structure	def, reusable code, parameters	Build functions to draw shapes	Student organizes code using functions
4	Colors & Patterns	pencolor, fillcolor, begin_fill	Create colorful patterns (stars, flowers)	Student applies creativity using color systems
5	Coordinates System	goto(x,y), positioning, direction	Build a scene (house, sun, trees)	Student controls object placement precisely
6	Turtle Race Game	random module, multiple objects, simple logic	Build Turtle Race game	Student creates a working mini-game
7	Independent Project	combining all concepts, debugging	Design own project (drawing/game)	Student builds a complete self-project
8	Final Project & Presentation	explanation, refinement, communication	Present project to class	Student explains logic + demonstrates project

---

🏆 Certification & Recognition

All students who complete **Python LEVEL 5: Turtle Graphics & Visual Systems** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certificate recognizes students' achievement in:

- Visual Programming & System Thinking
- Python Fundamentals (loops, functions, logic)
- Creative Coding & Design
- Problem-Solving & Debugging
- Independent Project Development
- Communication & Presentation Skills

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

- Students present their project (3–5 minutes)
- Explain how their system works (logic + structure)
- Demonstrate key features of their program

🔹 Code Explanation

- Students identify at least:
  - 1 loop used
  - 1 function created
  - 1 system they designed

🔹 Certification & Recognition Ceremony

- Awarding official CIY.CLUB Certificate
- Recognition of student progress and creativity
- Group reflection and celebration

🔥 CIY Insight (Core Philosophy Embedded)

Students don’t just learn how to draw with code
They learn how to think in systems, structure logic, and create with intention

Python LEVEL 6: Logic Extension & System Capability

CIY.CLUB Thailand — From Logic → Control → Real System

Learning Outcomes

By the end of this level, students will be able to:

- Apply advanced operators such as `%`, `//`, and `**` in practical problem-solving situations.
- Use logical operators (`and`, `or`, `not`) to create more complex and accurate decision-making systems.
- Use Python modules such as `random` and `datetime` to extend program functionality.
- Control loop behavior effectively with `break` and `continue`.
- Build programs that save and retrieve data using file handling.
- Handle runtime errors using `try` and `except` to make programs safer and more reliable.
- Design systems that are more robust, flexible, and aligned with real-world programming logic.

Course Highlights

This level helps students move beyond basic coding and begin thinking like real system builders. Instead of writing programs that only run once, students learn how to create programs that can make better decisions, control flow more effectively, remember data, and recover from errors.

Key highlights of this level include:

- **Logic Extension** — Students deepen their understanding of operators and conditions to build stronger logic systems.
- **System Control** — Students learn how to manage loops and program flow with greater precision.
- **Real Capability** — Students begin using modules to simulate real program features such as randomness and time-based functions.
- **Persistence** — Students learn how programs can save information using files.
- **Program Safety** — Students understand how error handling prevents crashes and improves user experience.

- **Real-World Thinking** — Students start building programs that behave more like practical applications rather than simple exercises.

Course Structure (Table)

Week	Focus Area	Topics Covered	Practical Application	Expected Outcome
Week 1	Advanced Operators	`%`, `//`, `**`	Even/odd checker, time conversion, exponential calculation	Students understand how advanced operators solve more meaningful math and logic problems
Week 2	Logical Operators & Loop Control	`and`, `or`, `not`, `break`, `continue`	Validation systems, controlled repetition, exit and skip logic	Students can build programs with multiple conditions and better control flow
Week 3	Python Modules	`random`, `datetime`	Random game logic, number generators, time-based output	Students can extend programs using built-in Python tools
Week 4	Integrated Logic Project	Combine operators, logic, loops, modules	Smart Number Checker System	Students apply extended logic in one complete mini system
Week 5	File Handling Basics	`open()`, `"r"`, `"w"`, `"a"`, read/write/append	Saving names, scores, logs, simple records	Students understand how programs store and retrieve information
Week 6	Error Handling	`try`, `except`	Preventing crashes from invalid input or file issues	Students can create safer, more reliable programs

Week 7	System Capability Project	File handling + error handling integration	Simple Data Logger System	Students build a program that saves user data and handles issues safely
Week 8	Final Review, Certification & Recognition	Project presentation, concept review, reflection	Showcase of Level 6 system projects	Students complete the level with recognition of their progress and system-building skills

---

Certification & Recognition

At the end of this level, students will be recognized for completing an important transition in their Python journey: moving from basic programming into controlled and reliable system design.

Students who complete **Python Level 6: Logic Extension & System Capability** will be able to demonstrate that they can:

- Build programs with stronger logical structure
- Manage conditions and loops with greater control
- Use modules to make programs more dynamic
- Save and retrieve information using files
- Protect programs from crashing through error handling
- Think more clearly about how real systems work in practice

This recognition reflects not only coding progress, but also the development of computational thinking, debugging awareness, and system responsibility.

Certification & Recognition (Week 8)

The final week (Week 8) will include Certification & Recognition.

During the final week, students will:

- Review the major concepts learned throughout the level
- Present or demonstrate their final mini system
- Reflect on how their programs became more controlled and reliable
- Receive recognition for completing Level 6

Possible recognition components may include:

- **Level Completion Certificate**
- **Project Showcase Presentation**
- **Coach Feedback Summary**
- **Skill Recognition in Logic, Control, and System Capability**

This final stage helps students feel that their work has real value and visible progress. It also reinforces the CIY.CLUB philosophy that learning should lead to practical ability, confidence, and meaningful outcomes.

Python LEVEL 7: Advanced Thinking & Data Systems

CIY.CLUB Thailand — From Control → Intelligence & Data Thinking

---

Learning Outcomes

By the end of this level, students will be able to:

- Manipulate and process text using advanced string operations such as `.lower()`, `.upper()`, slicing, and formatting.
- Design systems that validate, clean, and transform user input into structured data.
- Apply modular arithmetic (`%`) to create cyclic and transformation-based logic systems.
- Understand and implement pattern-based logic such as shifting, wrapping, and rotation.
- Use dictionaries to organize, store, and count data efficiently.
- Analyze data by grouping, counting, and ranking results.
- Build intelligent mini-systems that combine string processing, mathematical logic, and data handling.
- Clearly explain how their system processes input, transforms data, and produces meaningful output.

Course Highlights

This level marks a major transition from controlled programming into intelligent system thinking. Students no longer focus only on how programs run, but on how programs process information and generate insights.

Key highlights include:

- **String as a System**
  Students treat text as data that can be cleaned, transformed, and validated rather than just displayed.
- **Mathematical Logic in Coding**
  Modular arithmetic becomes a tool for building cyclic systems such as rotation, wrapping, and encryption logic.

- **Data Thinking & Analysis**
  Students move from storing data to analyzing patterns, counting occurrences, and generating results.
- **From Code → Insight**
  Programs are designed not just to execute instructions, but to interpret and transform information.
- **Bridge to AI Thinking**
  Students begin understanding that pattern + transformation = the foundation of intelligent systems.

Course Structure (Table)

Week	Focus Area	Topics Covered	Practical Application	Expected Outcome
Week 1	String Systems	`.lower()`, `.upper()`, slicing, `.strip()`	Text cleaning and transformation tasks	Students correctly process and transform input text
Week 2	String Processing	Input validation, formatting rules	Build input validation system (e.g., username/password)	Students can clean and validate user input effectively
Week 3	Modular Arithmetic	`%`, cyclic logic, wrapping systems	Rotation patterns, remainder-based logic	Students understand and apply wrap-around logic
Week 4	Encryption Logic	Caesar Cipher, shifting characters	Build text encryption/decryption system	Students can encode and decode text using logic
Week 5	Dictionary Basics	Key-value structure, counting patterns	Vote counting, frequency tracking	Students can store and organize structured data
Week 6	Data Analysis	Grouping, ranking, sorting insights	Analyze datasets and display results	Students generate meaningful insights from data
Week 7	Integrated System Project	Combine string + math + dictionary	Build full intelligent system	Students create end-to-end functional systems

Week 8	Presentation & Reflection	Explanation, system breakdown	Project presentation and review	Students clearly explain logic, data, and transformation

---

Certification & Recognition

At the completion of this level, students will be recognized for advancing into intelligent system thinking and data-driven programming.

Students who complete **Python Level 7: Advanced Thinking & Data Systems** will demonstrate that they can:

- Process and transform text as structured data
- Apply mathematical logic to control system behavior
- Organize and analyze data using dictionaries
- Build systems that generate insights rather than just outputs
- Combine multiple concepts into one integrated intelligent system
- Explain their logic clearly and confidently

Recognition areas include:

- **Advanced Logical Thinking**
- **Data Analysis & Pattern Recognition**
- **Text Processing Systems**
- **Mathematical Logic in Programming**
- **Problem Solving & System Design**

Certification & Recognition (Week 8)

The final week (Week 8) will include Certification & Recognition.

During the final week, students will participate in a structured showcase of their work and learning progression.

🔹 System Demonstration

Students will present a complete working system that includes:

- Input → Processing → Output flow
- Text transformation or data analysis functionality

- Clear system behavior and results

🔹 Logic Breakdown

Students will explain:

- How input text is processed and transformed
- How modular arithmetic (%) controls system logic
- How data is stored, counted, and analyzed using dictionaries

This reinforces understanding and builds confidence in communication.

🔹 Certification & Recognition Ceremony

Students will receive:

- **Certificate of Completion (Level 7)**
- Recognition of their system project
- Coach feedback on their logic and system design
- Acknowledgment of their progression into intelligent and data-driven thinking

Python LEVEL 8: Iteration & Object Systems

CIY.CLUB Thailand — From Data Thinking → Structured Systems & Objects

---

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Use for loops to iterate through sequences and control repeated logic precisely
- Apply range() and iteration patterns to build structured loops
- Process lists and dictionaries using iteration
- Understand how real systems handle collections of data step-by-step
- Understand objects as real-world entities (data + behavior)
- Create classes and objects using Python (class & instance)
- Define attributes and methods to model real-world systems
- Design programs using object-oriented thinking (entity + interaction)
- Combine iteration and object systems to build scalable programs
- Explain how their system processes data using loops and objects

---

🚀 Course Highlights

- **Iteration as a System**
  Students move from repeating actions → to processing structured data (lists, records, datasets)
- **From Variables → Objects**
  Programs evolve from simple variables into structured entities with behavior
- **Real-World Mapping**
  Students connect coding to real systems:
  - User → object
  - Product → object
  - Game character → object
- **Scalable System Design**
  Students stop writing repetitive code and begin designing reusable systems
- **Foundation for Real Programming & AI Systems**
  Iteration + Objects = core of backend systems, data pipelines, and AI structures

---

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Activity Format	Success Criteria
1	Introduction to Iteration	for loop, range(), iteration concept	Practice loops with numbers	Student controls loop execution
2	Iterating Data	loop through lists & dictionaries	Display and process data	Student processes structured data
3	List Operations	append(), remove(), len()	Build list manager system	Student modifies and manages data
4	Loop + Data System	condition + loop integration	Filtering & validation systems	Student builds data-processing system
5	Introduction to OOP	class, object, attributes	Create simple objects	Student understands object structure
6	Methods & Behavior	methods, self, interaction	Add behavior to objects	Student builds interactive objects
7	Object System Integration	multiple objects + loops	Manage object collections	Student builds object-based system
8	Final Project & Presentation	full system integration	Build and present system	Student explains system clearly

---

🏆 Certification & Recognition

All students who complete **Python LEVEL 8: Iteration & Object Systems** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certificate recognizes students' achievement in:

- Iteration Thinking — Processing structured data using loops
- Data Handling — Managing lists and dictionaries effectively
- Object-Oriented Thinking — Designing systems using objects
- System Design — Combining iteration and objects into scalable systems
- Problem Solving — Building structured and reusable logic
- Communication Skills — Explaining system logic clearly and confidently

---

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

- Students present a complete object-based system
- Demonstrate how data is processed using loops
- Show interaction between multiple objects

🔹 System Logic Explanation

Students explain:

- How iteration processes data step-by-step
- How objects represent real-world entities
- How their system is structured (data + behavior + interaction)

🔹 Certification & Recognition Ceremony

- Awarding official CIY.CLUB Certificate
- Recognition of structured system design and object thinking
- Reflection on progression from coding → system → structured architecture

Python LEVEL 9: Algorithm & Efficiency Thinking

CIY.CLUB Thailand — From System → Optimization & Problem Solving

---

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Understand algorithms as step-by-step problem-solving processes
- Compare multiple solutions to the same problem and identify which is more efficient
- Analyze program efficiency using step count and repetition (without formal Big-O)
- Design structured algorithms using loops and conditions
- Implement basic searching algorithms (linear search)
- Implement basic sorting algorithms (bubble sort, selection sort)
- Identify inefficient patterns such as unnecessary loops or repeated work
- Optimize programs by reducing steps and improving logic flow
- Explain why one solution is faster, simpler, or more efficient than another
- Apply algorithm thinking to real-world systems (ranking, filtering, processing data)

🚀 Course Highlights

- **From System → Optimization**
  Students move beyond building systems to improving how systems perform
- **Thinking in Steps**
  Every program is evaluated by:
  - Number of steps
  - Number of repetitions
- **Multiple Solutions Mindset**
  Students learn that:
  - One problem can have many solutions
  - Some solutions are more efficient
- **Efficiency Awareness**
  Students begin asking:
  - Can this be faster?
  - Can this be simpler?

- **Bridge to Computer Science Thinking**
  Introduces core ideas of:
  - Algorithm design
  - Performance thinking
  - Structured problem solving

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Activity Format	Success Criteria
1	What is an Algorithm	step-by-step logic, multiple solutions	Real-life task breakdown	Student explains process as steps
2	Efficiency Thinking	step count, comparing solutions	Compare slow vs fast methods	Student identifies better solution
3	Loop Efficiency	nested loops, repeated work	Analyze and reduce loops	Student improves inefficient code
4	Searching Algorithms	linear search	Find data in list system	Student builds search logic
5	Sorting Algorithms (1)	bubble sort	Step-by-step sorting	Student understands sorting process
6	Sorting Algorithms (2)	selection sort, optimization	Compare sorting methods	Student improves sorting logic
7	Optimization Project	refactoring, improving systems	Improve existing program	Student reduces steps and complexity
8	Final Project & Presentation	algorithm explanation + efficiency	Build optimized system	Student explains logic + improvement

---

🏆 Certification & Recognition

All students who complete **Python LEVEL 9: Algorithm & Efficiency Thinking** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certificate recognizes students' achievement in:

- Algorithm Thinking — Designing structured step-by-step solutions

- Efficiency Awareness — Evaluating and improving program performance
- Problem Solving — Comparing and selecting better approaches
- Data Processing — Searching and sorting structured data
- System Optimization — Improving existing systems logically
- Communication Skills — Explaining why solutions are effective

---

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

- Students present an optimized system (e.g., ranking system, sorting system)
- Demonstrate how their system processes and improves data

🔹 Algorithm & Efficiency Explanation

Students explain:

- Step-by-step algorithm used
- Difference between original vs improved solution
- How they reduced steps or improved performance

🔹 Certification & Recognition Ceremony

- Awarding official CIY.CLUB Certificate
- Recognition of algorithm thinking and optimization skills
- Reflection on progression from coding → system → structured → optimized thinking

Python LEVEL 10: Problem Solving Systems

CIY.CLUB Thailand — From Optimization → Real-World System Design

---

🎯 Learning Outcomes

By the end of this level, students will be able to:

- Break down real-world problems into structured system components
- Design complete systems using Input → Process → Output → Feedback
- Combine algorithms, data structures, and logic into real applications
- Apply searching, sorting, and optimization in meaningful contexts
- Handle real-world constraints (invalid input, edge cases, missing data)
- Design multi-step workflows (not just single scripts)
- Build systems that simulate real-world behavior (users, data, decisions)
- Test, debug, and refine systems through iteration
- Explain their system architecture clearly (not just code)
- Present solutions as structured systems, not just programs

🚀 Course Highlights

From Algorithm → Real Problem Solving

Students move from:
👉 “solve problem”
→ 👉 “design system to solve problem”

System Architecture Thinking

Students learn to think in layers:

- Input Layer
- Processing Layer
- Data Layer
- Output Layer

Real Constraints

Programs must handle:

- wrong input
- missing data
- unexpected behavior

Multi-Step Systems

No more single script
→ systems now include multiple steps, flows, and decisions

Builder Mindset

Students stop asking:
❌ “What code should I write?”

Start asking:
✔ “How should the system work?”

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Practical Application	Expected Outcome
1	Problem Breakdown	decomposition, system mapping	break real problem into steps	Students structure problems clearly
2	System Design Basics	IPO + feedback loop	design system flow diagram	Students design full system flow
3	Data Handling Systems	combine list, dict, objects	structured data system	Students manage complex data

4	Multi-Step Logic	chaining processes	multi-stage system	Students build workflow systems
5	Edge Cases & Validation	invalid input, error scenarios	robust input systems	Students handle real-world issues
6	System Integration	combine all components	full system build (part 1)	Students connect system layers
7	Optimization & Refinement	improve system performance	debug + optimize	Students refine system quality
8	Final Project & Presentation	full system explanation	present real system	Students explain architecture clearly

---

🧠 Key Concept Flow (CIY Thinking)

👉 Before (Level 9):
Algorithm → Efficiency

👉 Now (Level 10):
Problem → System → Solution → Refinement

🧪 Example Progression

🔻 Before (Simple Script)

```
score = int(input("Enter score: "))
if score >= 50:
  print("Pass")
else:
  print("Fail")
```

👉 Single logic only

🔻 After (System Thinking)

```
def get_score():
  try:
    return int(input("Enter score: "))
```

```
    except:
        return None

def evaluate(score):
    if score is None:
        return "Invalid"
    elif score >= 50:
        return "Pass"
    else:
        return "Fail"

def main():
    score = get_score()
    result = evaluate(score)
    print(result)

main()
```

👉 Student learns:

- modular system
- input validation
- structured flow

🎯 Final Project (System-Level)

Project: Real-World Management System

Students choose one:

1. Student Management System
2. Inventory System
3. Booking / Reservation System
4. Score Analysis System

System Requirements:

- Input system (user interaction)
- Data storage (list/dictionary/object)
- Processing logic (algorithm + condition)

- Output system (structured result)
- Error handling (invalid input)
- Optimization (efficient logic)

Expected Output:

👉 Not just code
👉 But system explanation:

- How data flows
- How decisions are made
- Why design choices were used

🏆 Certification & Recognition

Students who complete Python Level 10 will demonstrate:

- Problem Solving Thinking — Breaking problems into systems
- System Design — Structuring real-world applications
- Data Handling — Managing complex structured data
- Algorithm Application — Using logic effectively in real scenarios
- Robust Programming — Handling errors and edge cases
- Optimization — Improving system performance
- Communication — Explaining system architecture clearly

Website (design)

LEVEL 1: Web UI Components

CIY.CLUB Thailand — From Code → Visual Components

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Understand HTML as a structure for content (not decoration)
- Apply basic CSS to control visual appearance (color, font, spacing)
- Use the box model (margin, padding, border) to control layout
- Build reusable UI components such as cards, buttons, and sections
- Connect code to visual output and understand how design is constructed
- Organize content into clear and structured layouts
- Design simple user interfaces using component thinking
- Explain how UI elements are built and combined into a page

🚀 Course Highlights

- **Component Thinking (Core Concept)**
  Students learn that websites are built from reusable components, not random elements
- **Code → Visual Connection**
  Immediate feedback helps students understand how HTML and CSS affect the UI
- **Structured Design Approach**
  Students organize content into sections, cards, and layouts
- **Reusable UI Building**
  Instead of one-time design, students create components they can reuse
- **Foundation for Web Development**
  Prepares students for responsive design, frameworks, and real web systems

---

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Activity Format	Success Criteria

1	Introduction to HTML	structure, tags, head, body	Build simple page	Student creates structured HTML page
2	Text & Content	headings, paragraphs, images	Content layout practice	Student organizes content clearly
3	Introduction to CSS	color, font, styling basics	Style text and elements	Student applies basic styling
4	Box Model	margin, padding, border	Layout spacing practice	Student controls spacing properly
5	UI Components (1)	card structure	Build simple card	Student builds reusable card
6	UI Components (2)	buttons, sections	Build UI elements	Student creates interactive UI components
7	Component System	combine components	Build full layout	Student structures a full UI system
8	Final Project & Presentation	UI system + explanation	Present project	Student explains component design

---

🏆 Certification & Recognition

All students who complete **LEVEL 1: Web UI Components** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certificate recognizes students' achievement in:

- HTML Structure — Building well-organized web pages
- CSS Styling — Applying visual design using code
- UI Component Design — Creating reusable visual components
- Layout Thinking — Structuring content clearly and effectively
- Problem Solving — Building and improving UI systems
- Communication Skills — Explaining design and structure clearly

---

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

- Students present a complete UI system
- Demonstrate multiple card designs (profile, product, game)
- Show visual structure and layout

🔹 Component Explanation

Students explain:

- How their UI is divided into components
- How HTML structure supports the design
- How CSS controls appearance and layout

🔹 Certification & Recognition Ceremony

- Awarding official CIY.CLUB Certificate
- Recognition of UI design and component thinking
- Reflection on transition from code → visual system

LEVEL 2: Layout & Web Systems

CIY.CLUB Thailand — From Components → Full Website System

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Design multi-section websites with clear structure
- Use Flexbox to control layout and alignment of elements
- Understand user flow from top → middle → action
- Build navigation systems using menus and links
- Organize content into logical sections (hero, content, footer)
- Apply basic UX principles such as readability and spacing
- Connect multiple components into a complete website system
- Explain how a website is structured and how users interact with it

---

🚀 Course Highlights

- **From Components → System Thinking**
  Students move from individual UI elements → to full website systems
- **Layout Control with Flexbox**
  Learn how real websites align, space, and structure content
- **User Flow Understanding**
  Students design websites with a clear journey:
  Top (attention) → Middle (content) → Action (interaction)
- **Navigation as a System**
  Students build menus and links that connect sections/pages
- **Real Website Structure**
  Students build layouts similar to real-world websites

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Activity Format	Success Criteria

1	Layout Introduction	section structure, layout concept	Build multi-section page	Student understands page structure
2	Flexbox Basics	display flex, direction, alignment	Layout practice	Student controls layout using Flexbox
3	Advanced Flexbox	justify-content, align-items	Complex layout tasks	Student aligns elements properly
4	Navigation System	menu, links, anchor navigation	Build navbar	Student creates working navigation
5	Hero Section Design	headline, image, CTA	Build hero section	Student creates strong top section
6	Content Section	cards, layout structure	Build content area	Student organizes content clearly
7	Footer & System Integration	footer + full layout	Combine all sections	Student builds full website system
8	Final Project & Presentation	complete website + explanation	Present system	Student explains flow and structure

---

🏆 Certification & Recognition

Students who complete **LEVEL 2: Layout & Web Systems** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certification recognizes achievement in:

- Layout Design — Structuring full web pages using Flexbox
- Navigation Systems — Creating menus and page flow
- UI Integration — Combining components into full systems
- UX Thinking — Designing readable and structured user flow
- Problem Solving — Building functional multi-section websites
- Communication — Explaining layout and system structure clearly

---

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

Students present a complete multi-section website including:

- Navbar
- Hero section
- Content section (cards)
- Footer

🔹 System Flow Explanation

Students explain:

- How the website is structured (sections)
- How users navigate through the website
- How layout and spacing improve readability

🔹 Certification & Recognition Ceremony

- Certificate awarding
- Recognition of system thinking and layout design
- Reflection on progression from component → full website system

LEVEL 3: Web Product & Portfolio

CIY.CLUB Thailand — From Website → Real Product Thinking

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Design a complete personal or product-based website with clear purpose
- Organize content for real users (not just for display)
- Apply structure, layout, and UX principles together in one system
- Understand user perspective (who is this website for?)
- Create clear and readable web content (About, Projects, Contact)
- Build a portfolio that represents their work and identity
- Think like a creator — designing with intention, not just coding
- Present their website as a real digital product

🚀 Course Highlights

- **From Website → Product Thinking**
  Students move beyond building websites → to creating purposeful digital products
- **User-Centered Design**
  Students think about:
  - Who is the user?
  - What do they need?
- **Content as a System**
  Content is structured clearly:
  - About (identity)
  - Projects (value)
  - Contact (action)
- **UX Clarity & Readability**
  Focus on:
  - Clear layout
  - Easy navigation
  - Readable content
- **Portfolio as Real Output**
  Students finish with a usable portfolio they can show to others

---

📅 Course Structure (8 Weeks)

Week	Focus Area	Topics Covered	Activity Format	Success Criteria
1	Product Thinking	purpose, user, content planning	Plan website idea	Student defines purpose clearly
2	Content Structure	About, Projects, Contact	Content writing	Student organizes content logically
3	Layout Planning	section structure, flow	Wireframe design	Student plans page structure
4	UX Basics	readability, spacing, clarity	Improve layout	Student creates readable UI
5	Build Portfolio (1)	About section	Develop personal section	Student presents identity clearly
6	Build Portfolio (2)	Projects section	Showcase projects	Student displays work effectively
7	Build Portfolio (3)	Contact section + integration	Complete website	Student builds full product
8	Final Project & Presentation	portfolio + explanation	Present product	Student explains purpose and UX

🏆 Certification & Recognition

Students who complete **LEVEL 3: Web Product & Portfolio** will receive a **Certificate of Completion from CIY.CLUB Thailand**

This certification recognizes achievement in:

- Product Thinking — Designing websites with purpose and user focus
- Content Structuring — Organizing meaningful and clear information
- UX Design — Creating readable and user-friendly layouts
- Web Development — Building complete portfolio websites
- Problem Solving — Improving usability and structure
- Communication — Presenting a digital product clearly

🎤 The Final Week (Week 8) Will Include:

🔹 Final Project Presentation

Students present a complete personal/product website including:

- About Me section
- Projects showcase
- Contact section
- Clear layout and purpose

🔹 Product Explanation

Students explain:

- Who the website is for (target user)
- What the purpose of the website is
- How the structure supports user experience

🔹 Certification & Recognition Ceremony

- Certificate awarding
- Recognition of product thinking and portfolio creation
- Reflection on progression from code → website → real product

Website (interactive)

JS LEVEL 1: Interaction & Logic

CIY.CLUB Thailand — From Static → Interactive Thinking

Learning Outcomes

By the end of this course, students will be able to:

- Understand how JavaScript controls behavior on a webpage, moving from static content to interactive systems
- Use variables in JavaScript to store and manage data clearly, while recognizing the connection to Python-style variables
- Apply conditional logic using `if / else` to control decisions in a program
- Review and use loops to repeat actions efficiently
- Understand how the DOM connects JavaScript to visible webpage elements
- Create basic interactions such as button clicks, text changes, and simple user feedback
- Think in terms of Input → Process → Output when designing interactions
- Build a simple interactive webpage with clear purpose and user response

---

Course Highlights

From Static to Interactive
Students move beyond writing code as isolated logic and begin using JavaScript to control what happens on a real webpage. This helps them see programming as an active system rather than just written instructions.

Python to JavaScript Bridge
This course can connect well with Python learning because the logic is very similar even though the syntax changes. Variables, conditions, and loops still serve the same purpose, allowing students to transfer their thinking from Python into web programming more smoothly.

DOM as a Real Control System
Students learn that HTML provides structure, while JavaScript adds behavior. Through DOM interaction, they begin to understand how code can directly affect what users see and experience.

Event-Based Thinking
Unlike many beginner Python programs that run from top to bottom, JavaScript introduces the idea of waiting for user action. Students learn that interaction often follows the pattern: action happens first, then the program responds.

Visible and Meaningful Output
Instead of only printing results in a console, students create visible changes on the page itself. This makes coding feel more practical, immediate, and engaging.

Course Structure (Table)

Week	Theme	Key Concepts	Activity	Success Criteria
1	JS Basics & Variables	`let`, `const`, data types	Store and display values in the console	Students understand how JavaScript variables work
2	Condition Logic	`if / else`	Create simple decision-based outputs	Students can control flow using conditions
3	Loop Review	`for`, `while`	Repeat actions using loop structures	Students can apply loop logic correctly
4	Introduction to DOM	`document`, element selection	Select and connect HTML elements to JavaScript	Students understand how JS connects to webpage structure
5	DOM Manipulation	Change text, update content	Modify text and elements dynamically	Students can change webpage content through code
6	Events & Click Interaction	`onclick`, event-driven behavior	Build button-based response systems	Students can create interaction based on user action
7	Mini Project Build	Logic + DOM integration	Develop a mini interactive page	Students combine multiple concepts into one working system
8	Final Project + Presentation	Full interaction flow	Present a Button Click System or Mini Interactive Page	Students demonstrate a complete interactive webpage and explain how it works

---

Certification & Recognition

In the final week, students will complete and present their **Mini Interactive Web Project**. This project will serve as the main demonstration of their understanding of JavaScript interaction and logic.

During **Week 8**, students will participate in **Certification & Recognition**, where they will showcase their final work and explain how their webpage responds to user input. They will be encouraged to present their project using the structure of **Input** → **Process** → **Output**, helping them communicate both technical understanding and system thinking.

Certification will be based on the following areas:

- Ability to create a working interactive feature
- Clear use of logic such as variables, conditions, or loops
- Correct use of DOM interaction
- Understanding of how the system works
- Confidence in presenting the final project

This final stage helps students transition from simply writing code to creating purposeful digital interactions. It recognizes not only completion, but also growth in logic, structure, and real-world application.

JS LEVEL 2: Dynamic Web Systems

CIY.CLUB Thailand — From Interaction → System Thinking

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Understand how data (arrays / objects) drives dynamic behavior in a webpage
- Use arrays to manage lists of data (e.g., tasks, scores, items)
- Use objects to structure and organize related data
- Apply event systems to handle multiple user interactions
- Dynamically create, update, and remove content on the page
- Connect data + UI into a single working system
- Design simple logic systems (e.g., todo flow, scoring system)
- Think in terms of state → update → display
- Build interactive systems that respond to real user input continuously

---

🚀 Course Highlights

From Interaction → System Thinking
Students move beyond simple clicks and begin building systems where data changes over time and affects the UI dynamically.

Data as the Core System
Instead of hardcoding values, students learn that:

- arrays = collections of data
- objects = structured data
  👉 Data becomes the “brain” of the system

Dynamic Content Creation
Students learn to:

- create elements using JavaScript
- update UI based on data
- remove or modify content dynamically

👉 Webpages become “alive” and constantly changing

Event System Expansion
From single click → to multiple events:

- add item
- delete item
- update state

👉 Students design full interaction flows, not just single actions

Real System Thinking (State-Based)
Students understand:

- State = current data
- Action = user input
- Update = system changes
- Display = UI reflects state

👉 Foundation of real apps (React mindset later)

📅 Course Structure (Table)

Week	Theme	Key Concepts	Activity	Success Criteria
1	Arrays as Data System	array basics, push, pop	Manage list of items	Students can store and update multiple values
2	Objects for Structure	key-value pairs	Represent structured data	Students organize data logically
3	Event System Expansion	multiple events	Handle different user actions	Students control multiple interactions
4	Dynamic Content Creation	createElement, append	Add elements to page	Students generate UI from JS
5	Updating & Removing Content	edit / delete elements	Modify existing UI	Students control full lifecycle of elements
6	State-Based Thinking	data → UI sync	Track and update system state	Students understand state-driven behavior

7	Project Build	combine data + UI + events	Build todo / game system	Students create a working dynamic system
8	Final Project + Presentation	full integration	Todo List / Simple Game Logic	Students present complete system

---

🏆 Certification & Recognition

In the final week (Week 8), students will complete and present their **Dynamic Web System Project**.

Final Week Focus

Students will:

- Build and present either:
  - **Todo List System** (add / delete / manage tasks)
  - **Simple Game Logic System** (score, actions, updates)
- Demonstrate:
  - Data management using arrays / objects
  - Event handling for user interaction
  - Dynamic UI updates
- Explain their system using:
  👉 State → Action → Update → Display

Certification Criteria

Students will receive certification based on:

- System functionality (Does the system work correctly?)
- Data structure usage (arrays / objects used effectively)
- Interaction design (clear and usable UI flow)
- Logical thinking (clear connection between data and UI)
- Code understanding (ability to explain their system)

Recognition Focus

- From “Interactive Page” → “Dynamic System”
- From “Click Response” → “State Management”

- From “Single Action” → “Connected System Flow”

👉 Students transition from basic interaction to **real application thinking**

JS LEVEL 3: Web Application Thinking

CIY.CLUB Thailand — From Dynamic System → Application Thinking

---

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Understand how web applications manage user input and system flow
- Handle form data (input fields, buttons, submissions) effectively
- Apply validation logic to ensure correct and meaningful user input
- Design multi-step processes (step-by-step user journeys)
- Control system flow using conditions and structured logic
- Simulate real-world systems such as login and booking processes
- Connect user input → validation → system response → output
- Think in terms of user flow and application structure
- Build structured web applications with clear purpose and logic

---

🚀 Course Highlights

From System → Application Thinking
Students move beyond dynamic components and begin building full systems that simulate real-world applications such as booking and login systems.

User Flow as a System
Students learn to design:

- Step 1 → Input
- Step 2 → Validation
- Step 3 → Decision
- Step 4 → Output

👉 Focus shifts from “what happens” → “how the whole process flows”

Form Handling in Real Context
Students work with real input systems:

- text fields
- buttons
- submission flow

👉 Input becomes structured and meaningful

Validation as Control Layer
Students learn:

- required fields
- condition checks
- preventing invalid input

👉 System becomes more reliable and realistic

Multi-Step Logic (Process Thinking)
Students build systems where:

- actions depend on previous steps
- logic branches based on user input

👉 Foundation of real applications (auth systems, booking flows)

📅 Course Structure (Table)

Week	Theme	Key Concepts	Activity	Success Criteria
1	Form Basics	input fields, buttons	Capture user input	Students can collect data from users
2	Form Handling	events, submit logic	Process form data	Students can control form behavior
3	Validation Logic	required, conditions	Validate user input	Students prevent invalid data
4	Conditional Flow	if / else system flow	Decision-based systems	Students control outcomes based on input
5	Multi-Step Systems	step-by-step logic	Build multi-step process	Students design structured flows
6	Application Simulation	login / booking logic	Simulate real systems	Students connect full system flow
7	Project Build	integrate all concepts	Build full application	Students create a working system

8	Final Project + Presentation	full application system	Booking System / Login Simulation	Students present complete application

---

🏆 Certification & Recognition

In the final week (Week 8), students will complete and present their **Web Application Project**.

Final Week Focus

Students will:

- Build and present one of the following:
  - **Booking System** (input → validation → confirmation flow)
  - **Login Simulation** (username/password → validation → access result)
- Demonstrate:
  - Form handling and input control
  - Validation logic
  - Multi-step system flow
- Explain their system using:
  👉 Input → Validation → Process → Output

Certification Criteria

Students will receive certification based on:

- Functionality of the application (Does the flow work correctly?)
- Accuracy of validation (Handles correct vs incorrect input properly)
- Logical structure (Clear step-by-step system design)
- User flow clarity (Easy to understand and use)
- Code understanding (Can explain how the system works)

Recognition Focus

- From “Dynamic System” → “Application Builder”
- From “Data Handling” → “User Flow Design”
- From “Single Page Logic” → “Structured Application Thinking”

👉 Students transition into thinking like **real web application designers**

JS LEVEL 4: Mini Web Project Studio

CIY.CLUB Thailand — From Learning → Independent Creation

🎯 Learning Outcomes

By the end of this course, students will be able to:

- Review and reconnect their understanding of HTML, CSS, and JavaScript as one complete web system
- Identify how real websites combine structure, design, and interaction together
- Plan a small web project with clear purpose and user flow
- Apply HTML for content structure, CSS for visual design, and JavaScript for interaction
- Build a simple project independently with reduced step-by-step guidance
- Improve their project through testing, fixing, and refining
- Present their project clearly by explaining how the system works
- Think like a beginner web creator, not just a student following instructions

---

🚀 Course Highlights

Full Web Recap in One System
Students begin by revisiting HTML, CSS, and JavaScript together so they can see how all three work as one connected web-building system.

Real Website Observation
Before building, students look at simple real-world website examples and identify where structure, style, and interaction are being used. This helps them connect classroom learning to practical design.

Small Project Ownership
Instead of only following teacher-made exercises, students move into creating a small project of their own. This builds confidence, independence, and decision-making.

Build → Improve → Present
The course is structured like a real mini production cycle. Students review, observe, build, refine, and finally present their work as a finished web project.

Creator Mindset Development
This level helps students shift from “I can follow web lessons” to “I can make a small website project myself.”

📅 Course Structure (Table)

Week	Theme	Key Concepts	Activity	Success Criteria
1	HTML + CSS + JS Recap	structure, styling, interaction	Review how all three work together in one webpage	Students can explain the role of HTML, CSS, and JS clearly
2	Web Example Analysis	real website examples, structure, UI, interaction	Observe and discuss simple websites that use content, design, and interactivity	Students can identify structure, style, and interaction in real examples
3	Project Planning	project idea, page purpose, layout sketch	Choose a mini project and plan content and flow	Students create a clear idea and simple project plan
4	Build Phase 1	HTML structure, basic CSS	Start building the page layout and content	Students create the basic page structure successfully
5	Build Phase 2	styling, sections, layout improvement	Improve design and organize content clearly	Students build a readable and visually structured page
6	Build Phase 3	JavaScript interaction	Add simple interactivity such as buttons, toggles, or text changes	Students create at least one working interactive feature
7	Project Refinement	debugging, improvement, final polishing	Test, fix, and improve project quality	Students complete a functional and presentable mini project
8	Final Project Presentation	project explanation, system walkthrough	Present the final mini web project	Students explain how their webpage works and demonstrate their final result

---

🏆 Certification & Recognition

In the final week (Week 8), students will participate in **Certification & Recognition** through the presentation of their completed **Mini Web Project**.

During this final stage, students will present a small but complete website project that shows their understanding of structure, design, and interaction. They will be encouraged to explain how HTML, CSS, and JavaScript were used together to create the final result.

The final week will include:

- **Final Project Presentation**
  Students present their mini web project to show its purpose, layout, and interaction.
- **System Explanation**
  Students explain:
  - how the page is structured using HTML
  - how CSS improves the visual design
  - how JavaScript adds interactivity
- **Certification & Recognition Ceremony**
  Students receive recognition for completing the project and demonstrating growth in web design, logic, and presentation.

Certification Criteria

Students will be recognized based on:

- Completion of a working mini web project
- Clear use of HTML, CSS, and JavaScript together
- Ability to demonstrate at least one interactive feature
- Overall structure and visual clarity of the project
- Confidence in explaining their work during presentation

Recognition Focus

This level recognizes students for moving:

- from learning separate web skills → to combining them into one project
- from guided coding → to small independent creation
- from practice tasks → to real project presentation

แผนที่จะอัพต่อ

🔹 STEP ต่อไป: API

👉 ตอนนี้เด็กจะเข้า:

- request → response
- data flow
- external system

🎯 Project:

- weather app
- AI API call

🔹 STEP ต่อไป: Database (SQL / NoSQL)

👉 ตอนนี้เด็กจะเข้า:

- data storage
- persistent system

⚖️ SQL vs NoSQL (ตอบตรงๆ)

👉 สำหรับ CIY:

- เริ่มด้วย **NoSQL (ง่ายกว่า)**
- แล้วค่อยไป SQL

🧠 Insight สำคัญ (ระดับคุณ)

👉 Curriculum ที่ “โคตรแข็ง” จะเป็นแบบนี้:

Python → Thinking
Web → Structure
JavaScript → Interaction
API → Connection
Database → Memory

🔥 สรุปแบบไม่อวย

👉 ถ้าคุณถามว่า “อะไรต่อดี”

คำตอบคือ:

🥇 JavaScript (ต้องมา)

🥈 API (ตามมา)

🥉 Database (หลังสุด)

🚀 ถ้าจะเอาโคตรคม

ผมช่วยได้:

- 🔧 ออกแบบ **JS Curriculum (3 Levels) แบบ CIY Signature**
- 🚀 หรือวางทั้ง track:
  → JS → API → Database → AI Web App

แล้ว ไปผนวกการใช้ เว็บกับpython
ไปสาย pygame สร้างเกมขึ้นเว็บด้วย ไพตอน แล้วไป สายข้อมูล